



Standardizing Information and Communication Systems

C# Language Specification

Brief history

This ECMA Standard is based on a submission from Hewlett-Packard, Intel, and Microsoft, that describes a language called C#, which was developed within Microsoft. The principal inventors of this language were Anders Hejlsberg, Scott Wiltamuth, and Peter Golde. The first widely distributed implementation of C# was released by Microsoft in July 2000, as part of its .NET Framework initiative.

ECMA Technical Committee 39 (TC39) Task Group 2 (TG2) was formed in September 2000, to produce a standard for C#. Another Task Group, TG3, was also formed at that time to produce a standard for a library and execution environment called Common Language Infrastructure (CLI). (CLI is based on a subset of the .NET Framework.) Although Microsoft's implementation of C# relies on CLI for library and runtime support, other implementations of C# need not, provided they support an alternate way of getting at the minimum CLI features required by this C# standard.

As the definition of C# evolved, the goals used in its design were as follows:

- C# is intended to be a simple, modern, general-purpose, object-oriented programming language.
- The language, and implementations thereof, should provide support for software engineering principles such as strong type checking, array bounds checking, detection of attempts to use uninitialized variables, and automatic garbage collection. Software robustness, durability, and programmer productivity are important.
- The language is intended for use in developing software components suitable for deployment in distributed environments.
- Source code portability is very important, as is programmer portability, especially for those programmers already familiar with C and C++.
- Support for internationalization is very important.
- C# is intended to be suitable for writing applications for both hosted and embedded systems, ranging from the very large that use sophisticated operating systems, down to the very small having dedicated functions.
- Although C# applications are intended to be economical with regards to memory and processing power requirements, the language was not intended to compete directly on performance and size with C or assembly language.

The development of this standard started in November 2000.

It is intended that the final version of this ECMA Standard will be submitted to ISO/IEC JTC 1 for adoption under its fast-track procedure.

It is expected there will be future revisions to this Standard, primarily to add new functionality.

Adopted as an ECMA Standard by the General Assembly of December 2001.

Table of Contents

1		
2		
3	1. Scope	1
4	2. Conformance.....	3
5	3. References	5
6	4. Definitions	7
7	5. Notational conventions	9
8	6. Acronyms and abbreviations.....	11
9	7. General description	13
10	8. Language Overview.....	15
11	8.1 Getting started.....	15
12	8.2 Types.....	16
13	8.2.1 Predefined types	17
14	8.2.2 Conversions.....	19
15	8.2.3 Array types	20
16	8.2.4 Type system unification	21
17	8.3 Variables and parameters	22
18	8.4 Automatic memory management.....	25
19	8.5 Expressions	27
20	8.6 Statements.....	28
21	8.7 Classes	31
22	8.7.1 Constants.....	32
23	8.7.2 Fields.....	33
24	8.7.3 Methods.....	34
25	8.7.4 Properties.....	35
26	8.7.5 Events.....	36
27	8.7.6 Operators.....	37
28	8.7.7 Indexers	38
29	8.7.8 Instance constructors	39
30	8.7.9 Destructors	39
31	8.7.10 Static constructors	40
32	8.7.11 Inheritance.....	40
33	8.8 Structs	41
34	8.9 Interfaces.....	42
35	8.10 Delegates.....	43
36	8.11 Enums	44
37	8.12 Namespaces and assemblies	45
38	8.13 Versioning.....	46
39	8.14 Attributes	48
40	9. Lexical structure	51
41	9.1 Programs	51
42	9.2 Grammars.....	51
43	9.2.1 Lexical grammar	51
44	9.2.2 Syntactic grammar.....	51
45	9.3 Lexical analysis.....	52
46	9.3.1 Line terminators	52
47	9.3.2 Comments	52

1	9.3.3 White space	54
2	9.4 Tokens.....	54
3	9.4.1 Unicode escape sequences	54
4	9.4.2 Identifiers	55
5	9.4.3 Keywords	56
6	9.4.4 Literals.....	57
7	9.4.5 Operators and punctuators.....	62
8	9.5 Pre-processing directives	62
9	9.5.1 Conditional compilation symbols.....	63
10	9.5.2 Pre-processing expressions	63
11	9.5.3 Declaration directives.....	64
12	9.5.4 Conditional compilation directives	65
13	9.5.5 Diagnostic directives	67
14	9.5.6 Region control.....	67
15	9.5.7 Line directives	68
16	10. Basic concepts	69
17	10.1 Application startup.....	69
18	10.2 Application termination	69
19	10.3 Declarations	70
20	10.4 Members	72
21	10.4.1 Namespace members.....	72
22	10.4.2 Struct members.....	72
23	10.4.3 Enumeration members.....	73
24	10.4.4 Class members.....	73
25	10.4.5 Interface members	73
26	10.4.6 Array members.....	73
27	10.4.7 Delegate members	73
28	10.5 Member access.....	73
29	10.5.1 Declared accessibility.....	74
30	10.5.2 Accessibility domains	74
31	10.5.3 Protected access for instance members	77
32	10.5.4 Accessibility constraints.....	77
33	10.6 Signatures and overloading.....	78
34	10.7 Scopes	79
35	10.7.1 Name hiding	81
36	10.8 Namespace and type names	83
37	10.8.1 Fully qualified names	84
38	10.9 Automatic memory management.....	85
39	10.10 Execution order.....	87
40	11. Types.....	89
41	11.1 Value types	89
42	11.1.1 Default constructors	90
43	11.1.2 Struct types.....	90
44	11.1.3 Simple types	91
45	11.1.4 Integral types.....	91
46	11.1.5 Floating point types.....	92
47	11.1.6 The <code>decimal</code> type	93
48	11.1.7 The <code>bool</code> type.....	94
49	11.1.8 Enumeration types.....	94
50	11.2 Reference types.....	94
51	11.2.1 Class types.....	95
52	11.2.2 The <code>object</code> type.....	95
53	11.2.3 The <code>string</code> type.....	95

1	11.2.4 Interface types	95
2	11.2.5 Array types	96
3	11.2.6 Delegate types	96
4	11.3 Boxing and unboxing	96
5	11.3.1 Boxing conversions	96
6	11.3.2 Unboxing conversions	97
7	12. Variables.....	99
8	12.1 Variable categories.....	99
9	12.1.1 Static variables	99
10	12.1.2 Instance variables	99
11	12.1.3 Array elements	100
12	12.1.4 Value parameters.....	100
13	12.1.5 Reference parameters	100
14	12.1.6 Output parameters	100
15	12.1.7 Local variables	101
16	12.2 Default values	101
17	12.3 Definite assignment	102
18	12.3.1 Initially assigned variables	102
19	12.3.2 Initially unassigned variables	103
20	12.3.3 Precise rules for determining definite assignment	103
21	12.4 Variable references	111
22	12.5 Atomicity of variable references.....	111
23	13. Conversions	113
24	13.1 Implicit conversions.....	113
25	13.1.1 Identity conversion.....	113
26	13.1.2 Implicit numeric conversions	113
27	13.1.3 Implicit enumeration conversions	114
28	13.1.4 Implicit reference conversions	114
29	13.1.5 Boxing conversions	115
30	13.1.6 Implicit constant expression conversions.....	115
31	13.1.7 User-defined implicit conversions.....	115
32	13.2 Explicit conversions.....	115
33	13.2.1 Explicit numeric conversions	115
34	13.2.2 Explicit enumeration conversions	117
35	13.2.3 Explicit reference conversions	117
36	13.2.4 Unboxing conversions.....	118
37	13.2.5 User-defined explicit conversions	118
38	13.3 Standard conversions	118
39	13.3.1 Standard implicit conversions	118
40	13.3.2 Standard explicit conversions.....	118
41	13.4 User-defined conversions.....	119
42	13.4.1 Permitted user-defined conversions	119
43	13.4.2 Evaluation of user-defined conversions	119
44	13.4.3 User-defined implicit conversions.....	120
45	13.4.4 User-defined explicit conversions	120
46	14. Expressions.....	123
47	14.1 Expression classifications	123
48	14.1.1 Values of expressions.....	124
49	14.2 Operators.....	124
50	14.2.1 Operator precedence and associativity	124
51	14.2.2 Operator overloading.....	125
52	14.2.3 Unary operator overload resolution.....	126
53	14.2.4 Binary operator overload resolution.....	127

1	14.2.5 Candidate user-defined operators	127
2	14.2.6 Numeric promotions.....	127
3	14.3 Member lookup	129
4	14.3.1 Base types.....	129
5	14.4 Function members.....	130
6	14.4.1 Argument lists	132
7	14.4.2 Overload resolution	134
8	14.4.3 Function member invocation.....	136
9	14.5 Primary expressions	138
10	14.5.1 Literals.....	138
11	14.5.2 Simple names	138
12	14.5.3 Parenthesized expressions	140
13	14.5.4 Member access	140
14	14.5.5 Invocation expressions	142
15	14.5.6 Element access	143
16	14.5.7 This access	145
17	14.5.8 Base access.....	145
18	14.5.9 Postfix increment and decrement operators	146
19	14.5.10 The new operator.....	147
20	14.5.11 The typeof operator	151
21	14.5.12 The checked and unchecked operators	152
22	14.6 Unary expressions	154
23	14.6.1 Unary plus operator	154
24	14.6.2 Unary minus operator.....	154
25	14.6.3 Logical negation operator.....	155
26	14.6.4 Bitwise complement operator.....	155
27	14.6.5 Prefix increment and decrement operators	155
28	14.6.6 Cast expressions	156
29	14.7 Arithmetic operators	157
30	14.7.1 Multiplication operator.....	157
31	14.7.2 Division operator.....	158
32	14.7.3 Remainder operator	159
33	14.7.4 Addition operator	160
34	14.7.5 Subtraction operator	162
35	14.8 Shift operators	163
36	14.9 Relational and type-testing operators.....	164
37	14.9.1 Integer comparison operators	165
38	14.9.2 Floating-point comparison operators.....	166
39	14.9.3 Decimal comparison operators.....	166
40	14.9.4 Boolean equality operators.....	166
41	14.9.5 Enumeration comparison operators.....	167
42	14.9.6 Reference type equality operators	167
43	14.9.7 String equality operators	168
44	14.9.8 Delegate equality operators.....	168
45	14.9.9 The is operator.....	169
46	14.9.10 The as operator.....	169
47	14.10 Logical operators	170
48	14.10.1 Integer logical operators.....	170
49	14.10.2 Enumeration logical operators	170
50	14.10.3 Boolean logical operators.....	171
51	14.11 Conditional logical operators.....	171
52	14.11.1 Boolean conditional logical operators.....	171
53	14.11.2 User-defined conditional logical operators	172
54	14.12 Conditional operator	172
55	14.13 Assignment operators.....	173

1	14.13.1 Simple assignment.....	173
2	14.13.2 Compound assignment	175
3	14.13.3 Event assignment.....	176
4	14.14 Expression.....	176
5	14.15 Constant expressions.....	176
6	14.16 Boolean expressions.....	177
7	15. Statements	179
8	15.1 End points and reachability.....	179
9	15.2 Blocks	181
10	15.2.1 Statement lists	181
11	15.3 The empty statement	181
12	15.4 Labeled statements.....	182
13	15.5 Declaration statements	182
14	15.5.1 Local variable declarations.....	182
15	15.5.2 Local constant declarations	183
16	15.6 Expression statements.....	184
17	15.7 Selection statements.....	184
18	15.7.1 The <code>if</code> statement.....	184
19	15.7.2 The <code>switch</code> statement	185
20	15.8 Iteration statements	188
21	15.8.1 The <code>while</code> statement	188
22	15.8.2 The <code>do</code> statement.....	189
23	15.8.3 The <code>for</code> statement.....	189
24	15.8.4 The <code>foreach</code> statement	190
25	15.9 Jump statements.....	192
26	15.9.1 The <code>break</code> statement	193
27	15.9.2 The <code>continue</code> statement.....	194
28	15.9.3 The <code>goto</code> statement.....	194
29	15.9.4 The <code>return</code> statement	195
30	15.9.5 The <code>throw</code> statement	196
31	15.10 The <code>try</code> statement	197
32	15.11 The checked and unchecked statements.....	199
33	15.12 The <code>lock</code> statement	200
34	15.13 The <code>using</code> statement	200
35	16. Namespaces	203
36	16.1 Compilation units.....	203
37	16.2 Namespace declarations.....	203
38	16.3 Using directives	204
39	16.3.1 Using alias directives	205
40	16.3.2 Using namespace directives	207
41	16.4 Namespace members	208
42	16.5 Type declarations	209
43	17. Classes.....	211
44	17.1 Class declarations	211
45	17.1.1 Class modifiers.....	211
46	17.1.2 Class base specification.....	212
47	17.1.3 Class body	214
48	17.2 Class members	214
49	17.2.1 Inheritance.....	215
50	17.2.2 The <code>new</code> modifier	215
51	17.2.3 Access modifiers	216
52	17.2.4 Constituent types	216
53	17.2.5 Static and instance members	216

1	17.2.6 Nested types	217
2	17.2.7 Reserved member names.....	220
3	17.3 Constants.....	221
4	17.4 Fields.....	223
5	17.4.1 Static and instance fields	224
6	17.4.2 Readonly fields.....	224
7	17.4.3 Volatile fields	225
8	17.4.4 Field initialization	226
9	17.4.5 Variable initializers	227
10	17.5 Methods	229
11	17.5.1 Method parameters.....	230
12	17.5.2 Static and instance methods	235
13	17.5.3 Virtual methods	236
14	17.5.4 Override methods	238
15	17.5.5 Sealed methods.....	239
16	17.5.6 Abstract methods.....	240
17	17.5.7 External methods.....	241
18	17.5.8 Method body	241
19	17.5.9 Method overloading	242
20	17.6 Properties	242
21	17.6.1 Static and instance properties	243
22	17.6.2 Accessors.....	244
23	17.6.3 Virtual, sealed, override, and abstract accessors	248
24	17.7 Events.....	249
25	17.7.1 Field-like events	251
26	17.7.2 Event accessors	252
27	17.7.3 Static and instance events.....	253
28	17.7.4 Virtual, sealed, override, and abstract accessors	253
29	17.8 Indexers.....	254
30	17.8.1 Indexer overloading.....	257
31	17.9 Operators.....	257
32	17.9.1 Unary operators	258
33	17.9.2 Binary operators	259
34	17.9.3 Conversion operators.....	260
35	17.10 Instance constructors.....	261
36	17.10.1 Constructor initializers	262
37	17.10.2 Instance variable initializers.....	263
38	17.10.3 Constructor execution	263
39	17.10.4 Default constructors	264
40	17.10.5 Private constructors	265
41	17.10.6 Optional instance constructor parameters	265
42	17.11 Static constructors.....	266
43	17.12 Destructors	267
44	18. Structs.....	271
45	18.1 Struct declarations.....	271
46	18.1.1 Struct modifiers	271
47	18.1.2 Struct interfaces.....	271
48	18.1.3 Struct body	272
49	18.2 Struct members	272
50	18.3 Class and struct differences	272
51	18.3.1 Value semantics.....	272
52	18.3.2 Inheritance.....	273
53	18.3.3 Assignment.....	273
54	18.3.4 Default values.....	273

1	18.3.5 Boxing and unboxing	274
2	18.3.6 Meaning of <code>this</code>	274
3	18.3.7 Field initializers	274
4	18.3.8 Constructors	274
5	18.3.9 Destructors	275
6	18.4 Struct examples	275
7	18.4.1 Database integer type	275
8	18.4.2 Database boolean type	277
9	19. Arrays	279
10	19.1 Array types	279
11	19.1.1 The <code>System.Array</code> type	280
12	19.2 Array creation	280
13	19.3 Array element access	280
14	19.4 Array members	280
15	19.5 Array covariance	280
16	19.6 Array initializers	281
17	20. Interfaces	283
18	20.1 Interface declarations	283
19	20.1.1 Interface modifiers	283
20	20.1.2 Base interfaces	283
21	20.1.3 Interface body	284
22	20.2 Interface members	284
23	20.2.1 Interface methods	285
24	20.2.2 Interface properties	285
25	20.2.3 Interface events	286
26	20.2.4 Interface indexers	286
27	20.2.5 Interface member access	286
28	20.3 Fully qualified interface member names	288
29	20.4 Interface implementations	288
30	20.4.1 Explicit interface member implementations	289
31	20.4.2 Interface mapping	291
32	20.4.3 Interface implementation inheritance	293
33	20.4.4 Interface re-implementation	294
34	20.4.5 Abstract classes and interfaces	296
35	21. Enums	297
36	21.1 Enum declarations	297
37	21.2 Enum modifiers	297
38	21.3 Enum members	298
39	21.4 Enum values and operations	300
40	22. Delegates	301
41	22.1 Delegate declarations	301
42	22.2 Delegate instantiation	303
43	22.3 Delegate invocation	303
44	23. Exceptions	307
45	23.1 Causes of exceptions	307
46	23.2 The <code>System.Exception</code> class	307
47	23.3 How exceptions are handled	307
48	23.4 Common Exception Classes	308
49	24. Attributes	309
50	24.1 Attribute classes	309

1	24.1.1 Attribute usage	309
2	24.1.2 Positional and named parameters	310
3	24.1.3 Attribute parameter types	311
4	24.2 Attribute specification.....	311
5	24.3 Attribute instances	315
6	24.3.1 Compilation of an attribute	315
7	24.3.2 Run-time retrieval of an attribute instance	315
8	24.4 Reserved attributes.....	316
9	24.4.1 The <code>AttributeUsage</code> attribute.....	316
10	24.4.2 The <code>Conditional</code> attribute	316
11	24.4.3 The <code>Obsolete</code> attribute.....	318
12	25. Unsafe code.....	321
13	25.1 Unsafe contexts.....	321
14	25.2 Pointer types	323
15	25.3 Fixed and moveable variables.....	326
16	25.4 Pointer conversions.....	326
17	25.5 Pointers in expressions.....	327
18	25.5.1 Pointer indirection.....	328
19	25.5.2 Pointer member access	328
20	25.5.3 Pointer element access	329
21	25.5.4 The address-of operator.....	330
22	25.5.5 Pointer increment and decrement	330
23	25.5.6 Pointer arithmetic	331
24	25.5.7 Pointer comparison.....	332
25	25.5.8 The <code>sizeof</code> operator	332
26	25.6 The <code>fixed</code> statement	332
27	25.7 Stack allocation.....	336
28	25.8 Dynamic memory allocation.....	336
29	A. Grammar.....	339
30	A.1 Lexical grammar	339
31	A.1.1 Line terminators	339
32	A.1.2 White space	339
33	A.1.3 Comments.....	339
34	A.1.4 Tokens	340
35	A.1.5 Unicode character escape sequences.....	340
36	A.1.6 Identifiers	340
37	A.1.7 Keywords	341
38	A.1.8 Literals.....	342
39	A.1.9 Operators and punctuators.....	343
40	A.1.10 Pre-processing directives.....	344
41	A.2 Syntactic grammar	345
42	A.2.1 Basic concepts	345
43	A.2.2 Types	346
44	A.2.3 Variables.....	347
45	A.2.4 Expressions.....	347
46	A.2.5 Statements	350
47	A.2.6 Classes	354
48	A.2.7 Structs.....	359
49	A.2.8 Arrays	360
50	A.2.9 Interfaces	360
51	A.2.10 Enums.....	361
52	A.2.11 Delegates	362
53	A.2.12 Attributes.....	362

1	A.3 Grammar extensions for unsafe code.....	363
2	B. Portability issues	365
3	B.1 Undefined behavior.....	365
4	B.2 Implementation-defined behavior	365
5	B.3 Unspecified behavior.....	366
6	B.4 Other Issues	366
7	C. Naming guidelines.....	367
8	C.1 Capitalization styles	367
9	C.1.1 Pascal casing.....	367
10	C.1.2 Camel casing.....	367
11	C.1.3 All uppercase	367
12	C.1.4 Capitalization summary	367
13	C.2 Word choice	368
14	C.3 Namespaces.....	368
15	C.4 Classes.....	369
16	C.5 Interfaces	369
17	C.6 Enums.....	370
18	C.7 Static fields.....	371
19	C.8 Parameters	371
20	C.9 Methods.....	371
21	C.10 Properties.....	371
22	C.11 Events.....	372
23	C.12 Case sensitivity.....	372
24	C.13 Avoiding type name confusion	373
25	D. Standard Library.....	375
26	E. Documentation Comments.....	439
27	E.1 Introduction	439
28	E.2 Recommended tags.....	440
29	E.2.1 <c>	440
30	E.2.2 <code>	441
31	E.2.3 <example>	441
32	E.2.4 <exception>	441
33	E.2.5 <list>	442
34	E.2.6 <para>	443
35	E.2.7 <param>	443
36	E.2.8 <paramref>	444
37	E.2.9 <permission>	444
38	E.2.10 <remarks>	444
39	E.2.11 <returns>.....	445
40	E.2.12 <see>.....	445
41	E.2.13 <seealso>	445
42	E.2.14 <summary>	446
43	E.2.15 <value>	446
44	E.3 Processing the documentation file.....	446
45	E.3.1 ID string format	447
46	E.3.2 ID string examples.....	447
47	E.4 An example.....	450
48	E.4.1 C# source code.....	450
49	E.4.2 Resulting XML	453
50	F. Index.....	457

1. Scope

This clause is informative.

This ECMA Standard specifies the form and establishes the interpretation of programs written in the C# programming language. It specifies

- The representation of C# programs;
- The syntax and constraints of the C# language;
- The semantic rules for interpreting C# programs;
- The restrictions and limits imposed by a conforming implementation of C#.

This ECMA Standard does not specify

- The mechanism by which C# programs are transformed for use by a data-processing system;
- The mechanism by which C# applications are invoked for use by a data-processing system;
- The mechanism by which input data are transformed for use by a C# application;
- The mechanism by which output data are transformed after being produced by a C# application;
- The size or complexity of a program and its data that will exceed the capacity of any specific data-processing system or the capacity of a particular processor;
- All minimal requirements of a data-processing system that is capable of supporting a conforming implementation.

End of informative text.

2. Conformance

Conformance is of interest to the following audiences:

- Those designing, implementing, or maintaining C# implementations.
- Governmental or commercial entities wishing to procure C# implementations.
- Testing organizations wishing to provide a C# conformance test suite.
- Programmers wishing to port code from one C# implementation to another.
- Educators wishing to teach Standard C#.
- Authors wanting to write about Standard C#.

As such, conformance is most important, and the bulk of this ECMA Standard is aimed at specifying the characteristics that make C# implementations and C# programs conforming ones.

The text in this ECMA Standard that specifies requirements is considered *normative*. All other text in this specification is *informative*; that is, for information purposes only. Unless stated otherwise, all text is normative. Normative text is further broken into *required* and *conditional* categories. *Conditionally normative* text specifies requirements for a feature such that if that feature is provided, its syntax and semantics must be exactly as specified.

If any requirement of this ECMA Standard is violated, the behavior is undefined. Undefined behavior is otherwise indicated in this ECMA Standard by the words “undefined behavior” or by the omission of any explicit definition of behavior. There is no difference in emphasis among these three; they all describe “behavior that is undefined.”

A *strictly conforming program* shall use only those features of the language specified in this ECMA Standard as being required. (This means that a strictly conforming program cannot use any conditionally normative feature.) It shall not produce output dependent on any unspecified, undefined, or implementation-defined behavior.

A *conforming implementation* of C# must accept any strictly conforming program.

A conforming implementation of C# must provide and support all the types, values, objects, properties, methods, and program syntax and semantics described in this ECMA Standard.

A conforming implementation of C# shall interpret characters in conformance with the Unicode Standard, Version 3.0 or later, and ISO/IEC 10646-1. Conforming implementations must accept Unicode source files encoded with the UTF-8 encoding form.

A conforming implementation of C# shall not successfully translate source containing a `#error` preprocessing directive unless it is part of a group skipped by conditional compilation.

A conforming implementation of C# shall produce at least one diagnostic message if the source program violates any rule of syntax, or any negative requirement (defined as a “shall” or “shall not” or “error” or “warning” requirement), unless that requirement is marked with the words “no diagnostic is required”.

A conforming implementation of C# is permitted to provide additional types, values, objects, properties, and methods beyond those described in this ECMA Standard, provided they do not alter the behavior of any strictly conforming program. Conforming implementations are required to diagnose programs that use extensions that are ill formed according to this ECMA Standard. Having done so, however; they can compile and execute such programs. (The ability to have extensions implies that a conforming implementation reserves no identifiers other than those explicitly reserved in this ECMA Standard.)

C# LANGUAGE SPECIFICATION

- 1 A conforming implementation of C# shall be accompanied by a document that defines all implementation-
2 defined characteristics, and all extensions.
- 3 A conforming implementation of C# shall support the class library documented in §D. This library is
4 included by reference in this ECMA Standard.
- 5 A *conforming program* is one that is acceptable to a conforming implementation. (Such a program may
6 contain extensions or conditionally normative features.)

3. References

The following normative documents contain provisions, which, through reference in this text, constitute provisions of this ECMA Standard. For dated references, subsequent amendments to, or revisions of, any of these publications do not apply. However, parties to agreements based on this ECMA Standard are encouraged to investigate the possibility of applying the most recent editions of the normative documents indicated below. For undated references, the latest edition of the normative document referred to applies. Members of ISO and IEC maintain registers of currently valid ECMA Standards.

ECMA-335, 1st Edition, December 2001, *Common Language Infrastructure (CLI), Partition IV: Base Class Library (BCL), Extended Numerics Library, and Extended Array Library*.

ISO 31.11:1992, *Quantities and units — Part 11: Mathematical signs and symbols for use in the physical sciences and technology*.

ISO/IEC 2382.1:1993, *Information technology — Vocabulary — Part 1: Fundamental terms*.

ISO/IEC 10646 (all parts), *Information technology — Universal Multiple-Octet Coded Character Set (UCS)*.

IEC 60559:1989, *Binary floating-point arithmetic for microprocessor systems* (previously designated IEC 559:1989). (This standard is widely known by its U.S. national designation, ANSI/IEEE Standard 754-1985, *IEEE Standard for Binary Floating-Point Arithmetic*.) Due to the extremely widespread recognition of *IEEE* as the name of a form of floating-point representation and arithmetic, this ECMA Standard uses that term instead of its IEC equivalent.

The Unicode Consortium. The Unicode Standard, Version 3.0, defined by: *The Unicode Standard, Version 3.0* (Reading, MA, Addison-Wesley, 2000. ISBN 0-201-61633-5), and Unicode Technical Report #15: *Unicode Normalization Forms*.

The following references are informative:

ISO/IEC 9899:1999, *Programming languages — C*.

ISO/IEC 14882:1998, *Programming languages — C++*.

ANSI X3.274-1996, *Programming Language REXX*. (This document is useful in understanding floating-point decimal arithmetic rules.)

End of informative references

4. Definitions

For the purposes of this ECMA Standard, the following definitions apply. Other terms are defined where they appear in *italic* type or on the left side of a syntax rule. Terms explicitly defined in this ECMA Standard are not to be presumed to refer implicitly to similar terms defined elsewhere. Terms not defined in this ECMA Standard are to be interpreted according to ISO/IEC 2382.1. Mathematical symbols not defined in this ECMA Standard are to be interpreted according to ISO 31.11.

Application — refers to an assembly that has an entry point (§10.1). When an application is run, a new application domain is created. Several different instantiations of an application may exist on the same machine at the same time, and each has its own application domain.

Application domain — an entity that enables application isolation by acting as a container for application state. An application domain acts as a container and boundary for the types defined in the application and the class libraries it uses. Types loaded into one application domain are distinct from the same type loaded into another application domain, and instances of objects are not directly shared between application domains. For instance, each application domain has its own copy of static variables for these types, and a static constructor for a type is run at most once per application domain. Implementations are free to provide implementation-specific policy or mechanisms for the creation and destruction of application domains.

Argument — an expression in the comma-separated list bounded by the parentheses in a method or instance constructor call expression. It is also known as an *actual argument*.

Assembly — refers to one or more files that are output by the compiler as a result of program compilation. An assembly is a configured set of loadable code modules and other resources that together implement a unit of functionality. An assembly may contain types, the executable code used to implement these types, and references to other assemblies. The physical representation of an assembly is not defined by this specification. Essentially, an assembly is the output of the compiler.

Behavior — external appearance or action.

Behavior, implementation-defined — unspecified behavior where each implementation documents how the choice is made.

Behavior, undefined — behavior, upon use of a nonportable or erroneous construct or of erroneous data, for which this ECMA Standard imposes no requirements. [Possible handling of undefined behavior ranges from ignoring the situation completely with unpredictable results, to behaving during translation or execution in a documented manner characteristic of the environment (with or without the issuance of a diagnostic message), to terminating a translation or execution (with the issuance of a diagnostic message)].

Behavior, unspecified — behavior where this ECMA Standard provides two or more possibilities and imposes no further requirements on which is chosen in any instance.

Class library — refers to an assembly that can be used by other assemblies. Use of a class library does not cause the creation of a new application domain. Instead, a class library is loaded into the application domain that uses it. For instance, when an application uses a class library, that class library is loaded into the application domain for that application. If an application uses a class library A that itself uses a class library B, then both A and B are loaded into the application domain for the application.

Diagnostic message — a message belonging to an implementation-defined subset of the implementation's output messages.

Error, compile-time — an error reported during program translation.

Exception — an error condition that is outside the ordinary expected behavior.

- 1 **Implementation** — particular set of software (running in a particular translation environment under
2 particular control options) that performs translation of programs for, and supports execution of methods in, a
3 particular execution environment.
- 4 **Namespace** — a logical organizational system that provides a way of presenting program elements that are
5 exposed to other programs.
- 6 **Parameter** — a variable declared as part of a method, instance constructor, or indexer definition, which
7 acquires a value on entry to that method. It is also known as *formal parameter*.
- 8 **Program** — refers to one or more source files that are presented to the compiler. Essentially, a program is
9 the input to the compiler.
- 10 **Program, valid** — a C# program constructed according to the syntax rules and diagnosable semantic rules.
- 11 **Program instantiation** — the execution of an application.
- 12 **Recommended practice** — specification that is strongly recommended as being aligned with the intent of
13 the standard, but that may be impractical for some implementations
- 14 **Source file** — an ordered sequence of Unicode characters. Source files typically have a one-to-one
15 correspondence with files in a file system, but this correspondence is not required.
- 16 **Unsafe code** — code that is permitted to perform such lower-level operations as declaring and operating on
17 pointers, performing conversions between pointers and integral types, and taking the address of variables.
18 Such operations provide functionality such as permitting interfacing with the underlying operating system,
19 accessing a memory-mapped device, or implementing a time-critical algorithm.
- 20 **Warning, compile-time** — an informational message reported during program translation, that is intended
21 to identify a potentially questionable usage of a program element.

5. Notational conventions

Lexical and syntactic grammars for C# are interspersed throughout this specification. The lexical grammar defines how characters can be combined to form *tokens* (§9.4), the minimal lexical elements of the language. The syntactic grammar defines how tokens can be combined to make valid C# programs.

Grammar productions include both non-terminal and terminal symbols. In grammar productions, *non-terminal* symbols are shown in italic type, and **terminal** symbols are shown in a fixed-width font. Each non-terminal is defined by a set of productions. The first line of a set of productions is the name of the non-terminal, followed by a colon. Each successive indented line contains the right-hand side for a production that has the non-terminal symbol as the left-hand side. For example:

class-modifier:

```
new
public
protected
internal
private
abstract
sealed
```

defines the *class-modifier* non-terminal as having seven productions.

Alternatives are normally listed on separate lines, as shown above, though in cases where there are many alternatives, the phrase “one of” precedes a list of the options. This is simply shorthand for listing each of the alternatives on a separate line. For example:

decimal-digit: one of

```
0 1 2 3 4 5 6 7 8 9
```

is equivalent to:

decimal-digit:

```
0
1
2
3
4
5
6
7
8
9
```

A subscripted suffix “_{opt}”, as in *identifier_{opt}*, is used as shorthand to indicate an optional symbol. The example:

for-statement:

```
for ( for-initializeropt ; for-conditionopt ; for-iteratoropt ) embedded-statement
```

is equivalent to:

```
1      for-statement:  
2          for ( ; ; ) embedded-statement  
3          for ( for-initializer ; ; ) embedded-statement  
4          for ( ; for-condition ; ) embedded-statement  
5          for ( ; ; for-iterator ) embedded-statement  
6          for ( for-initializer ; for-condition ; ) embedded-statement  
7          for ( ; for-condition ; for-iterator ) embedded-statement  
8          for ( for-initializer ; ; for-iterator ) embedded-statement  
9          for ( for-initializer ; for-condition ; for-iterator ) embedded-statement
```

10

11 All terminal characters are to be understood as the appropriate Unicode character from the ASCII range, as
12 opposed to any similar-looking characters from other Unicode ranges.

6. Acronyms and abbreviations

This clause is informative.

The following acronyms and abbreviations are used throughout this ECMA Standard:

BCL — Base Class Library, which provides types to represent the built-in data types of the CLI, simple file access, custom attributes, security attributes, string manipulation, formatting, streams, and collections.

CLI — Common Language Infrastructure

CLS — Common Language Specification

IEC — the International Electrotechnical Commission

IEEE — the Institute of Electrical and Electronics Engineers

ISO — the International Organization for Standardization

The name C# is pronounced “C Sharp”.

The name C# is written as the LATIN CAPITAL LETTER C (U+0043) followed by the NUMBER SIGN # (U+000D).

End of informative text.

7. General description

This clause is informative.

This ECMA Standard is intended to be used by implementers, academics, and application programmers. As such, it contains a considerable amount of explanatory material that, strictly speaking, is not necessary in a formal language specification.

This standard is divided into the following subdivisions:

1. Front matter (clauses 1–7);
2. Language overview (clause 8);
3. The language syntax, constraints, and semantics (clauses 9–25);
4. Annexes

Examples are provided to illustrate possible forms of the constructions described. References are used to refer to related clauses. Notes are provided to give advice or guidance to implementers or programmers.

Annexes provide additional information and summarize the information contained in this ECMA Standard.

Clauses 2–5, 9–24, the beginning of 25, and the beginning of D form a normative part of this standard; all of clause 25 with the exception of the beginning is conditionally normative; and Brief history, clauses 1, 6–8, annexes A, B, C, and most of D, notes, examples, and the index are informative.

Except for whole clauses or annexes that are identified as being informative, informative text that is contained within normative text is indicated in two ways:

1. [*Example*: The following example ... code fragment, possibly with some narrative ... *end example*]
2. [*Note*: narrative ... *end note*]

End of informative text.

8. Language Overview

This clause is informative.

C# (pronounced “C Sharp”) is a simple, modern, object oriented, and type-safe programming language. It will immediately be familiar to C and C++ programmers. C# combines the high productivity of Rapid Application Development (RAD) languages and the raw power of C++.

The rest of this chapter describes the essential features of the language. While later chapters describe rules and exceptions in a detail-oriented and sometimes mathematical manner, this chapter strives for clarity and brevity at the expense of completeness. The intent is to provide the reader with an introduction to the language that will facilitate the writing of early programs and the reading of later chapters.

8.1 Getting started

The canonical “hello, world” program can be written as follows:

```
using System;
class Hello
{
    static void Main() {
        Console.WriteLine("hello, world");
    }
}
```

The source code for a C# program is typically stored in one or more text files with a file extension of `.cs`, as in `hello.cs`. Using a command-line compiler, such a program can be compiled with a command line like

```
csc hello.cs
```

which produces an application named `hello.exe`. The output produced by this application when it is run is:

```
hello, world
```

Close examination of this program is illuminating:

- The `using System;` directive references a namespace called `System` that is provided by the Common Language Infrastructure (CLI) class library. This namespace contains the `Console` class referred to in the `Main` method. Namespaces provide a hierarchical means of organizing the elements of one or more programs. A using-directive enables unqualified use of the types that are members of the namespace. The “hello, world” program uses `Console.WriteLine` as shorthand for `System.Console.WriteLine`.
- The `Main` method is a member of the class `Hello`. It has the `static` modifier, and so it is a method on the class `Hello` rather than on instances of this class.
- The entry point for an application—the method that is called to begin execution—is always a static method named `Main`.
- The “hello, world” output is produced using a class library. This standard does not include a class library. Instead, it references the class library provided by CLI.

For C and C++ developers, it is interesting to note a few things that do *not* appear in the “hello, world” program.

- The program does not use a global method for `Main`. Methods and variables are not supported at the global level; such elements are always contained within type declarations (e.g., class and struct declarations).
- The program does not use either “:.” or “->” operators. The “:.” is not an operator at all, and the “->” operator is used in only a small fraction of programs (which involve unsafe code). The separator “.” is used in compound names such as `Console.WriteLine`.
- The program does not contain forward declarations. Forward declarations are never needed, as declaration order is not significant.
- The program does not use `#include` to import program text. Dependencies among programs are handled symbolically rather than textually. This approach eliminates barriers between applications written using multiple languages. For example, the `Console` class need not be written in C#.

8.2 Types

C# supports two kinds of types: **value types** and **reference types**. Value types include simple types (e.g., `char`, `int`, and `float`), enum types, and struct types. Reference types include class types, interface types, delegate types, and array types.

Value types differ from reference types in that variables of the value types directly contain their data, whereas variables of the reference types store references to objects. With reference types, it is possible for two variables to reference the same object, and thus possible for operations on one variable to affect the object referenced by the other variable. With value types, the variables each have their own copy of the data, and it is not possible for operations on one to affect the other.

The example

```
using System;

class Class1
{
    public int value = 0;
}

class Test
{
    static void Main() {
        int val1 = 0;
        int val2 = val1;
        val2 = 123;

        Class1 ref1 = new Class1();
        Class1 ref2 = ref1;
        ref2.value = 123;

        Console.WriteLine("Values: {0}, {1}", val1, val2);
        Console.WriteLine("Refs: {0}, {1}", ref1.value, ref2.value);
    }
}
```

shows this difference. The output produced is

```
values: 0, 123
Refs: 123, 123
```

The assignment to the local variable `val1` does not impact the local variable `val2` because both local variables are of a value type (the type `int`) and each local variable of a value type has its own storage. In contrast, the assignment `ref2.value = 123`; affects the object that both `ref1` and `ref2` reference.

The lines

```
Console.WriteLine("Values: {0}, {1}", val1, val2);
Console.WriteLine("Refs: {0}, {1}", ref1.value, ref2.value);
```

deserve further comment, as they demonstrate some of the string formatting behavior of `Console.WriteLine`, which, in fact, takes a variable number of arguments. The first argument is a string,

which may contain numbered placeholders like {0} and {1}. Each placeholder refers to a trailing argument with {0} referring to the second argument, {1} referring to the third argument, and so on. Before the output is sent to the console, each placeholder is replaced with the formatted value of its corresponding argument.

Developers can define new value types through enum and struct declarations, and can define new reference types via class, interface, and delegate declarations. The example

```

using System;

public enum Color
{
    Red, Blue, Green
}

public struct Point
{
    public int x, y;
}

public interface IBase
{
    void F();
}

public interface IDerived: IBase
{
    void G();
}

public class A
{
    protected virtual void H() {
        Console.WriteLine("A.H");
    }
}

public class B: A, IDerived
{
    public void F() {
        Console.WriteLine("B.F, implementation of IDerived.F");
    }

    public void G() {
        Console.WriteLine("B.G, implementation of IDerived.G");
    }

    override protected void H() {
        Console.WriteLine("B.H, override of A.H");
    }
}

public delegate void EmptyDelegate();

```

shows an example of each kind of type declaration. Later sections describe type declarations in detail.

8.2.1 Predefined types

C# provides a set of predefined types, most of which will be familiar to C and C++ developers.

The predefined reference types are `object` and `string`. The type `object` is the ultimate base type of all other types. The type `string` is used to represent Unicode string values. Values of type `string` are immutable.

The predefined value types include signed and unsigned integral types, floating-point types, and the types `bool`, `char`, and `decimal`. The signed integral types are `sbyte`, `short`, `int`, and `long`; the unsigned integral types are `byte`, `ushort`, `uint`, and `ulong`; and the floating-point types are `float` and `double`.

The `bool` type is used to represent boolean values: values that are either true or false. The inclusion of `bool` makes it easier to write self-documenting code, and also helps eliminate the all-too-common C++ coding error in which a developer mistakenly uses “=” when “==” should have been used. In C#, the example

```

1      int i = ...;
2      F(i);
3      if (i = 0)  // Bug: the test should be (i == 0)
4          G();

```

5 results in a compile-time error because the expression `i = 0` is of type `int`, and `if` statements require an
6 expression of type `bool`.

7 The `char` type is used to represent Unicode characters. A variable of type `char` represents a single 16-bit
8 Unicode character.

9 The `decimal` type is appropriate for calculations in which rounding errors caused by floating point
10 representations are unacceptable. Common examples include financial calculations such as tax computations
11 and currency conversions. The `decimal` type provides 28 significant digits.

12 The table below lists the predefined types, and shows how to write literal values for each of them.

13

Type	Description	Example
<code>object</code>	The ultimate base type of all other types	<code>object o = null;</code>
<code>string</code>	String type; a string is a sequence of Unicode characters	<code>string s = "hello";</code>
<code>sbyte</code>	8-bit signed integral type	<code>sbyte val = 12;</code>
<code>short</code>	16-bit signed integral type	<code>short val = 12;</code>
<code>int</code>	32-bit signed integral type	<code>int val = 12;</code>
<code>long</code>	64-bit signed integral type	<code>long val1 = 12;</code> <code>long val2 = 34L;</code>
<code>byte</code>	8-bit unsigned integral type	<code>byte val1 = 12;</code>
<code>ushort</code>	16-bit unsigned integral type	<code>ushort val1 = 12;</code>
<code>uint</code>	32-bit unsigned integral type	<code>uint val1 = 12;</code> <code>uint val2 = 34U;</code>
<code>ulong</code>	64-bit unsigned integral type	<code>ulong val1 = 12;</code> <code>ulong val2 = 34U;</code> <code>ulong val3 = 56L;</code> <code>ulong val4 = 78UL;</code>
<code>float</code>	Single-precision floating point type	<code>float val = 1.23F;</code>
<code>double</code>	Double-precision floating point type	<code>double val1 = 1.23;</code> <code>double val2 = 4.56D;</code>
<code>bool</code>	Boolean type; a <code>bool</code> value is either true or false	<code>bool val1 = true;</code> <code>bool val2 = false;</code>
<code>char</code>	Character type; a <code>char</code> value is a Unicode character	<code>char val = 'h';</code>
<code>decimal</code>	Precise decimal type with 28 significant digits	<code>decimal val = 1.23M;</code>

14

15 Each of the predefined types is shorthand for a system-provided type. For example, the keyword `int` refers
16 to the struct `System.Int32`. As a matter of style, use of the keyword is favored over use of the complete
17 system type name.

18 Predefined value types such as `int` are treated specially in a few ways but are for the most part treated
19 exactly like other structs. Operator overloading enables developers to define new struct types that behave
20 much like the predefined value types. For instance, a `Digit` struct can support the same mathematical
21 operations as the predefined integral types, and can define conversions between `Digit` and predefined
22 types.

The predefined types employ operator overloading themselves. For example, the comparison operators `==` and `!=` have different semantics for different predefined types:

- Two expressions of type `int` are considered equal if they represent the same integer value.
- Two expressions of type `object` are considered equal if both refer to the same object, or if both are `null`.
- Two expressions of type `string` are considered equal if the string instances have identical lengths and identical characters in each character position, or if both are `null`.

The example

```
using System;
class Test
{
    static void Main() {
        string s = "Test";
        string t = string.Copy(s);
        Console.WriteLine(s == t);
        Console.WriteLine((object)s == (object)t);
    }
}
```

produces the output

```
True
False
```

because the first comparison compares two expressions of type `string`, and the second comparison compares two expressions of type `object`.

8.2.2 Conversions

The predefined types also have predefined conversions. For instance, conversions exist between the predefined types `int` and `long`. C# differentiates between two kinds of conversions: **implicit conversions** and **explicit conversions**. Implicit conversions are supplied for conversions that can safely be performed without careful scrutiny. For instance, the conversion from `int` to `long` is an implicit conversion. This conversion always succeeds, and never results in a loss of information. The following example

```
using System;
class Test
{
    static void Main() {
        int intValue = 123;
        long longValue = intValue;
        Console.WriteLine("{0}, {1}", intValue, longValue);
    }
}
```

implicitly converts an `int` to a `long`.

In contrast, explicit conversions are performed with a cast expression. The example

```
using System;
class Test
{
    static void Main() {
        long longValue = Int64.MaxValue;
        int intValue = (int) longValue;
        Console.WriteLine("(int) {0} = {1}", longValue, intValue);
    }
}
```

uses an explicit conversion to convert a `long` to an `int`. The output is:

```
(int) 9223372036854775807 = -1
```

because an overflow occurs. Cast expressions permit the use of both implicit and explicit conversions.

8.2.3 Array types

Arrays may be single-dimensional or multi-dimensional. Both “rectangular” and “jagged” arrays are supported.

Single-dimensional arrays are the most common type. The example

```
using System;
class Test
{
    static void Main() {
        int[] arr = new int[5];
        for (int i = 0; i < arr.Length; i++)
            arr[i] = i * i;
        for (int i = 0; i < arr.Length; i++)
            Console.WriteLine("arr[{0}] = {1}", i, arr[i]);
    }
}
```

creates a single-dimensional array of `int` values, initializes the array elements, and then prints each of them out. The output produced is:

```
arr[0] = 0
arr[1] = 1
arr[2] = 4
arr[3] = 9
arr[4] = 16
```

The type `int[]` used in the previous example is an array type. Array types are written using a non-array-type followed by one or more rank specifiers. The example

```
class Test
{
    static void Main() {
        int[] a1;           // single-dimensional array of int
        int[,] a2;          // 2-dimensional array of int
        int[,,] a3;         // 3-dimensional array of int
        int[][] j2;         // "jagged" array: array of (array of int)
        int[,,][,,] j3;     // array of (array of (array of int))
    }
}
```

shows a variety of local variable declarations that use array types with `int` as the element type.

Array types are reference types, and so the declaration of an array variable merely sets aside space for the reference to the array. Array instances are actually created via array initializers and array creation expressions. The example

```
class Test
{
    static void Main() {
        int[] a1 = new int[] {1, 2, 3};
        int[,] a2 = new int[,] {{1, 2, 3}, {4, 5, 6}};
        int[,,] a3 = new int[10, 20, 30];
        int[][] j2 = new int[3][];
        j2[0] = new int[] {1, 2, 3};
        j2[1] = new int[] {1, 2, 3, 4, 5, 6};
        j2[2] = new int[] {1, 2, 3, 4, 5, 6, 7, 8, 9};
    }
}
```

shows a variety of array creation expressions. The variables `a1`, `a2` and `a3` denote **rectangular arrays**, and the variable `j2` denotes a **jagged array**. It should be no surprise that these terms are based on the shapes of the arrays. Rectangular arrays always have a rectangular shape. Given the length of each dimension of the array, its rectangular shape is clear. For example, the lengths of `a3`’s three dimensions are 10, 20, and 30, respectively, and it is easy to see that this array contains 10*20*30 elements.

In contrast, the variable `j2` denotes a “jagged” array, or an “array of arrays”. Specifically, `j2` denotes an array of an array of `int`, or a single-dimensional array of type `int[]`. Each of these `int[]` variables can be initialized individually, and this allows the array to take on a jagged shape. The example gives each of the `int[]` arrays a different length. Specifically, the length of `j2[0]` is 3, the length of `j2[1]` is 6, and the length of `j2[2]` is 9.

[Note: In C++, an array declared as `int x[3][5][7]` would be considered a three dimensional rectangular array, while in C#, the declaration `int[][][]` declares a jagged array type. *end note*]

The element type and shape of an array—including whether it is jagged or rectangular, and the number of dimensions it has—are part of its type. On the other hand, the size of the array—as represented by the length of each of its dimensions—is not part of an array’s type. This split is made clear in the language syntax, as the length of each dimension is specified in the array creation expression rather than in the array type. For instance the declaration

```
int[, ,] a3 = new int[10, 20, 30];
```

has an array type of `int[, ,]` and an array creation expression of `new int[10, 20, 30]`.

For local variable and field declarations, a shorthand form is permitted so that it is not necessary to re-state the array type. For instance, the example

```
int[] a1 = new int[] {1, 2, 3};
```

can be shortened to

```
int[] a1 = {1, 2, 3};
```

without any change in program semantics.

The context in which an array initializer such as `{1, 2, 3}` is used determines the type of the array being initialized. The example

```
class Test
{
    static void Main() {
        short[] a = {1, 2, 3};
        int[] b = {1, 2, 3};
        long[] c = {1, 2, 3};
    }
}
```

shows that the same array initializer syntax can be used for several different array types. Because context is required to determine the type of an array initializer, it is not possible to use an array initializer in an expression context without explicitly stating the type of the array.

8.2.4 Type system unification

C# provides a “unified type system”. All types—including value types—derive from the type `object`. It is possible to call object methods on any value, even values of “primitive” types such as `int`. The example

```
using System;
class Test
{
    static void Main() {
        Console.WriteLine(3.ToString());
    }
}
```

calls the object-defined `ToString` method on an integer literal, resulting in the output “3”.

The example

```

1      class Test
2      {
3          static void Main() {
4              int i = 123;
5              object o = i;    // boxing
6              int j = (int) o; // unboxing
7          }
8      }

```

9 is more interesting. An `int` value can be converted to `object` and back again to `int`. This example shows
 10 both **boxing** and **unboxing**. When a variable of a value type needs to be converted to a reference type, an
 11 object **box** is allocated to hold the value, and the value is copied into the box. **Unboxing** is just the opposite.
 12 When an object box is cast back to its original value type, the value is copied out of the box and into the
 13 appropriate storage location.

14 This type system unification provides value types with the benefits of object-ness without introducing
 15 unnecessary overhead. For programs that don't need `int` values to act like objects, `int` values are simply
 16 32-bit values. For programs that need `int` values to behave like objects, this capability is available on
 17 demand. This ability to treat value types as objects bridges the gap between value types and reference types
 18 that exists in most languages. For example, a `Stack` class can provide `Push` and `Pop` methods that take and
 19 return `object` values.

```

20      public class Stack
21      {
22          public object Pop() {...}
23          public void Push(object o) {...}
24      }

```

25 Because C# has a unified type system, the `Stack` class can be used with elements of any type, including
 26 value types like `int`.

27 8.3 Variables and parameters

28 **Variables** represent storage locations. Every variable has a type that determines what values can be stored in
 29 the variable. **Local variables** are variables that are declared in methods, properties, or indexers. A local
 30 variable is defined by specifying a type name and a declarator that specifies the variable name and an
 31 optional initial value, as in:

```

32      int a;
33      int b = 1;

```

34 but it is also possible for a local variable declaration to include multiple declarators. The declarations of `a`
 35 and `b` can be rewritten as:

```

36      int a, b = 1;

```

37 A variable must be assigned before its value can be obtained. The example

```

38      class Test
39      {
40          static void Main() {
41              int a;
42              int b = 1;
43              int c = a + b; // error, a not yet assigned
44              ...
45          }
46      }

```

47 results in a compile-time error because it attempts to use the variable `a` before it is assigned a value. The
 48 rules governing definite assignment are defined in §12.3.

49 A **field** (§17.4) is a variable that is associated with a class or struct, or an instance of a class or struct. A field
 50 declared with the `static` modifier defines a **static variable**, and a field declared without this modifier
 51 defines an **instance variable**. A static field is associated with a type, whereas an instance variable is
 52 associated with an instance. The example

```

1      using Personnel.Data;
2      class Employee
3      {
4          private static DataSet ds;
5          public string Name;
6          public decimal Salary;
7      } ...
8  
```

9 shows an `Employee` class that has a private static variable and two public instance variables.

10 Formal parameter declarations also define variables. There are four kinds of parameters: value parameters,
11 reference parameters, output parameters, and parameter arrays.

12 A **value parameter** is used for “in” parameter passing, in which the value of an argument is passed into a
13 method, and modifications of the parameter do not impact the original argument. A value parameter refers to
14 its own variable, one that is distinct from the corresponding argument. This variable is initialized by copying
15 the value of the corresponding argument. The example

```

16      using System;
17      class Test {
18          static void F(int p) {
19              Console.WriteLine("p = {0}", p);
20              p++;
21          }
22          static void Main() {
23              int a = 1;
24              Console.WriteLine("pre: a = {0}", a);
25              F(a);
26              Console.WriteLine("post: a = {0}", a);
27          }
28      }

```

29 shows a method `F` that has a value parameter named `p`. The example produces the output:

```

30      pre: a = 1
31      p = 1
32      post: a = 1

```

33 even though the value parameter `p` is modified.

34 A **reference parameter** is used for “by reference” parameter passing, in which the parameter acts as an alias
35 for a caller-provided argument. A reference parameter does not itself define a variable, but rather refers to
36 the variable of the corresponding argument. Modifications of a reference parameter impact the
37 corresponding argument. A reference parameter is declared with a `ref` modifier. The example

```

38      using System;
39      class Test {
40          static void Swap(ref int a, ref int b) {
41              int t = a;
42              a = b;
43              b = t;
44          }
45          static void Main() {
46              int x = 1;
47              int y = 2;
48
49              Console.WriteLine("pre: x = {0}, y = {1}", x, y);
50              Swap(ref x, ref y);
51              Console.WriteLine("post: x = {0}, y = {1}", x, y);
52          }
53      }

```

54 shows a `Swap` method that has two reference parameters. The output produced is:

```

55      pre: x = 1, y = 2
56      post: x = 2, y = 1

```

The **ref** keyword must be used in both the declaration of the formal parameter and in uses of it. The use of **ref** at the call site calls special attention to the parameter, so that a developer reading the code will understand that the value of the argument could change as a result of the call.

An **output parameter** is similar to a reference parameter, except that the initial value of the caller-provided argument is unimportant. An output parameter is declared with an **out** modifier. The example

```

using System;
class Test {
    static void Divide(int a, int b, out int result, out int remainder) {
        result = a / b;
        remainder = a % b;
    }

    static void Main() {
        for (int i = 1; i < 10; i++)
            for (int j = 1; j < 10; j++) {
                int ans, r;
                Divide(i, j, out ans, out r);
                Console.WriteLine("{0} / {1} = {2}r{3}", i, j, ans, r);
            }
    }
}

```

shows a **Divide** method that includes two output parameters—one for the result of the division and another for the remainder.

For value, reference, and output parameters, there is a one-to-one correspondence between caller-provided arguments and the parameters used to represent them. A **parameter array** enables a many-to-one relationship: many arguments can be represented by a single parameter array. In other words, parameter arrays enable variable length argument lists.

A parameter array is declared with a **params** modifier. There can be only one parameter array for a given method, and it must always be the last parameter specified. The type of a parameter array is always a single dimensional array type. A caller can either pass a single argument of this array type, or any number of arguments of the element type of this array type. For instance, the example

```

using System;
class Test
{
    static void F(params int[] args) {
        Console.WriteLine("# of arguments: {0}", args.Length);
        for (int i = 0; i < args.Length; i++)
            Console.WriteLine("\targs[{0}] = {1}", i, args[i]);
    }

    static void Main() {
        F();
        F(1);
        F(1, 2);
        F(1, 2, 3);
        F(new int[] {1, 2, 3, 4});
    }
}

```

shows a method **F** that takes a variable number of **int** arguments, and several invocations of this method. The output is:

```

1      # of arguments: 0
2      # of arguments: 1
3      args[0] = 1
4      # of arguments: 2
5      args[0] = 1
6      args[1] = 2
7      # of arguments: 3
8      args[0] = 1
9      args[1] = 2
10     args[2] = 3
11     # of arguments: 4
12     args[0] = 1
13     args[1] = 2
14     args[2] = 3
15     args[3] = 4

```

Most of the examples presented in this introduction use the `WriteLine` method of the `Console` class. The argument substitution behavior of this method, as exhibited in the example

```

18     int a = 1, b = 2;
19     Console.WriteLine("a = {0}, b = {1}", a, b);

```

is accomplished using a parameter array. The `WriteLine` method provides several overloaded methods for the common cases in which a small number of arguments are passed, and one method that uses a parameter array.

```

23     namespace System
24     {
25         public class Console
26         {
27             public static void WriteLine(string s) {...}
28             public static void WriteLine(string s, object a) {...}
29             public static void WriteLine(string s, object a, object b) {...}
30             ...
31             public static void WriteLine(string s, params object[] args) {...}
32         }
33     }

```

8.4 Automatic memory management

Manual memory management requires developers to manage the allocation and de-allocation of blocks of memory. Manual memory management can be both time-consuming and difficult. In C#, **automatic memory management** is provided so that developers are freed from this burdensome task. In the vast majority of cases, automatic memory management increases code quality and enhances developer productivity without negatively impacting either expressiveness or performance.

The example

```

41     using System;
42     public class Stack
43     {
44         private Node first = null;
45         public bool Empty {
46             get {
47                 return (first == null);
48             }
49         }
50         public object Pop() {
51             if (first == null)
52                 throw new Exception("Can't Pop from an empty Stack.");
53             else {
54                 object temp = first.Value;
55                 first = first.Next;
56                 return temp;
57             }
58         }

```

```

1      public void Push(object o) {
2          first = new Node(o, first);
3      }
4      class Node
5      {
6          public Node Next;
7          public object Value;
8          public Node(object value): this(value, null) {}
9          public Node(object value, Node next) {
10             Next = next;
11             Value = value;
12         }
13     }
14 }

```

shows a **Stack** class implemented as a linked list of **Node** instances. **Node** instances are created in the **Push** method and are garbage collected when no longer needed. A **Node** instance becomes eligible for garbage collection when it is no longer possible for any code to access it. For instance, when an item is removed from the **Stack**, the associated **Node** instance becomes eligible for garbage collection.

The example

```

20     class Test
21     {
22         static void Main() {
23             Stack s = new Stack();
24             for (int i = 0; i < 10; i++)
25                 s.Push(i);
26             s = null;
27         }
28     }

```

shows code that uses the **Stack** class. A **Stack** is created and initialized with 10 elements, and then assigned the value **null**. Once the variable **s** is assigned **null**, the **Stack** and the associated 10 **Node** instances become eligible for garbage collection. The garbage collector is permitted to clean up immediately, but is not required to do so.

The garbage collector underlying C# may work by moving objects around in memory, but this motion is invisible to most C# developers. For developers who are generally content with automatic memory management but sometimes need fine-grained control or that extra bit of performance, C# provides the ability to write “unsafe” code. Such code can deal directly with pointer types and object addresses, however, C# requires the programmer to *fix* objects to temporarily prevent the garbage collector from moving them.

This “unsafe” code feature is in fact a “safe” feature from the perspective of both developers and users. Unsafe code must be clearly marked in the code with the modifier **unsafe**, so developers can't possibly use unsafe language features accidentally, and the compiler and the execution engine work together to ensure that unsafe code cannot masquerade as safe code. These restrictions limit the use of unsafe code to situations in which the code is trusted.

The example

```

44     using System;
45     class Test
46     {
47         static void writeLocations(byte[] arr) {
48             unsafe {
49                 fixed (byte* pArray = arr) {
50                     byte* pElem = pArray;
51                     for (int i = 0; i < arr.Length; i++) {
52                         byte value = *pElem;
53                         Console.WriteLine("arr[{0}] at 0x{1:X} is {2}",
54                             i, (uint)pElem, value);
55                         pElem++;
56                     }
57                 }
58             }
59         }

```



```

1      static void Main() {
2          byte[] arr = new byte[] {1, 2, 3, 4, 5};
3          writeLocations(arr);
4      }
5  }

```

shows an unsafe block in a method named `writeLocations` that fixes an array instance and uses pointer manipulation to iterate over the elements. The index, value, and location of each array element are written to the console. One possible example of output is:

```

9      arr[0] at 0x8E0360 is 1
10     arr[1] at 0x8E0361 is 2
11     arr[2] at 0x8E0362 is 3
12     arr[3] at 0x8E0363 is 4
13     arr[4] at 0x8E0364 is 5

```

but, of course, the exact memory locations may be different in different executions of the application.

8.5 Expressions

C# includes unary operators, binary operators, and one ternary operator. The following table summarizes the operators, listing them in order of precedence from highest to lowest:

Section	Category	Operators
14.5	Primary	<code>x.y</code> <code>f(x)</code> <code>a[x]</code> <code>x++</code> <code>x--</code> <code>new</code> <code>typeof</code> <code>checked</code> <code>unchecked</code>
0	Unary	<code>+</code> <code>-</code> <code>!</code> <code>~</code> <code>++x</code> <code>--x</code> <code>(T)x</code>
14.7	Multiplicative	<code>*</code> <code>/</code> <code>%</code>
14.7	Additive	<code>+</code> <code>-</code>
0	Shift	<code><<</code> <code>>></code>
14.9	Relational and type-testing	<code><</code> <code>></code> <code><=</code> <code>>=</code> <code>is</code> <code>as</code>
14.9	Equality	<code>==</code> <code>!=</code>
14.10	Logical AND	<code>&</code>
14.10	Logical XOR	<code>^</code>
14.10	Logical OR	<code> </code>
14.11	Conditional AND	<code>&&</code>
14.11	Conditional OR	<code> </code>
14.12	Conditional	<code>?:</code>
14.13	Assignment	<code>=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>+=</code> <code>-=</code> <code><<=</code> <code>>>=</code> <code>&=</code> <code>^=</code> <code> =</code>

When an expression contains multiple operators, the **precedence** of the operators controls the order in which the individual operators are evaluated. For example, the expression `x + y * z` is evaluated as `x + (y * z)` because the `*` operator has higher precedence than the `+` operator.

When an operand occurs between two operators with the same precedence, the **associativity** of the operators controls the order in which the operations are performed:

- Except for the assignment operators, all binary operators are **left-associative**, meaning that operations are performed from left to right. For example, $x + y + z$ is evaluated as $(x + y) + z$.
 - The assignment operators and the conditional operator ($?:$) are **right-associative**, meaning that operations are performed from right to left. For example, $x = y = z$ is evaluated as $x = (y = z)$.
- Precedence and associativity can be controlled using parentheses. For example, $x + y * z$ first multiplies y by z and then adds the result to x , but $(x + y) * z$ first adds x and y and then multiplies the result by z .

8.6 Statements

C# borrows most of its statements directly from C and C++, though there are some noteworthy additions and modifications. The table below lists the kinds of statements that can be used, and provides an example for each.

Statement	Example
Statement lists and block statements	<pre>static void Main() { F(); G(); { H(); I(); } }</pre>
Labeled statements and goto statements	<pre>static void Main(string[] args) { if (args.Length == 0) goto done; Console.WriteLine(args.Length); done: Console.WriteLine("Done"); }</pre>
Local constant declarations	<pre>static void Main() { const float pi = 3.14f; const int r = 123; Console.WriteLine(pi * r * r); }</pre>
Local variable declarations	<pre>static void Main() { int a; int b = 2, c = 3; a = 1; Console.WriteLine(a + b + c); }</pre>
Expression statements	<pre>static int F(int a, int b) { return a + b; } static void Main() { F(1, 2); // Expression statement }</pre>
if statements	<pre>static void Main(string[] args) { if (args.Length == 0) Console.WriteLine("No args"); else Console.WriteLine("Args"); }</pre>
switch statements	<pre>static void Main(string[] args) { switch (args.Length) { case 0: Console.WriteLine("No args"); break; case 1: Console.WriteLine("One arg "); break; default: int n = args.Length; Console.WriteLine("{0} args", n); break; } }</pre>
while statements	<pre>static void Main(string[] args) { int i = 0; while (i < args.Length) { Console.WriteLine(args[i]); i++; } }</pre>

do statements	<pre>static void Main() { string s; do { s = Console.ReadLine(); } while (s != "Exit"); }</pre>
for statements	<pre>static void Main(string[] args) { for (int i = 0; i < args.Length; i++) Console.WriteLine(args[i]); }</pre>
foreach statements	<pre>static void Main(string[] args) { foreach (string s in args) Console.WriteLine(s); }</pre>
break statements	<pre>static void Main(string[] args) { int i = 0; while (true) { if (i == args.Length) break; Console.WriteLine(args[i++]); } }</pre>
continue statements	<pre>static void Main(string[] args) { int i = 0; while (true) { Console.WriteLine(args[i++]); if (i < args.Length) continue; break; } }</pre>
return statements	<pre>static int F(int a, int b) { return a + b; } static void Main() { Console.WriteLine(F(1, 2)); return; }</pre>
throw statements and try statements	<pre>static int F(int a, int b) { if (b == 0) throw new Exception("Divide by zero"); return a / b; } static void Main() { try { Console.WriteLine(F(5, 0)); } catch (Exception e) { Console.WriteLine("Error"); } }</pre>
checked and unchecked statements	<pre>static void Main() { int x = Int32.MaxValue; Console.WriteLine(x + 1); // Overflow checked { Console.WriteLine(x + 1); // Exception } unchecked { Console.WriteLine(x + 1); // Overflow } }</pre>

lock statements	<pre>static void Main() { A a = ...; lock(a) { a.P = a.P + 1; } }</pre>
using statements	<pre>static void Main() { using (Resource r = new Resource()) { r.F(); } }</pre>

1

2 8.7 Classes

3 Class declarations define new reference types. A class can inherit from another class, and can implement
4 interfaces.

5 Class members can include constants, fields, methods, properties, events, indexers, operators, instance
6 constructors, destructors, static constructors, and nested type declarations. Each member has an associated
7 accessibility (§10.5), which controls the regions of program text that are able to access the member. There
8 are five possible forms of accessibility. These are summarized in the table below.

9

Form	Intuitive meaning
public	Access not limited
protected	Access limited to the containing class or types derived from the containing class
internal	Access limited to this program
protected internal	Access limited to this program or types derived from the containing class
private	Access limited to the containing type

10

11 The example

```

12     using System;
13     class MyClass
14     {
15         public MyClass() {
16             Console.WriteLine("Instance constructor");
17         }
18         public MyClass(int value) {
19             MyField = value;
20             Console.WriteLine("Instance constructor");
21         }
22         ~MyClass() {
23             Console.WriteLine("Destructor");
24         }
25         public const int MyConst = 12;
26         public int MyField = 34;
27         public void MyMethod(){
28             Console.WriteLine("MyClass.MyMethod");
29         }
30         public int MyProperty {
31             get {
32                 return MyField;
33             }

```

```

1      set {
2          MyField = value;
3      }
4  }
5  public int this[int index] {
6      get {
7          return 0;
8      }
9      set {
10         Console.WriteLine("this[{0}] = {1}", index, value);
11     }
12 }
13 public event EventHandler MyEvent;
14 public static MyClass operator+(MyClass a, MyClass b) {
15     return new MyClass(a.MyField + b.MyField);
16 }
17 internal class MyNestedClass
18 {}
19 }

```

shows a class that contains each kind of member. The example

```

21 class Test
22 {
23     static void Main() {
24         // Instance constructor usage
25         MyClass a = new MyClass();
26         MyClass b = new MyClass(123);
27
28         // Constant usage
29         Console.WriteLine("MyConst = {0}", MyClass.MyConst);
30
31         // Field usage
32         a.MyField++;
33         Console.WriteLine("a.MyField = {0}", a.MyField);
34
35         // Method usage
36         a.MyMethod();
37
38         // Property usage
39         a.MyProperty++;
40         Console.WriteLine("a.MyProperty = {0}", a.MyProperty);
41
42         // Indexer usage
43         a[3] = a[1] = a[2];
44         Console.WriteLine("a[3] = {0}", a[3]);
45
46         // Event usage
47         a.MyEvent += new EventHandler(MyHandler);
48
49         // Overloaded operator usage
50         MyClass c = a + b;
51     }
52
53     static void MyHandler(object sender, EventArgs e) {
54         Console.WriteLine("Test.MyHandler");
55     }
56
57     internal class MyNestedClass
58     {}
59 }

```

shows uses of these members.

8.7.1 Constants

A **constant** is a class member that represents a constant value: a value that can be computed at compile-time. Constants are permitted to depend on other constants within the same program as long as there are no circular dependencies. The rules governing constant expressions are defined in §14.15. The example

```

1      class Constants
2      {
3          public const int A = 1;
4          public const int B = A + 1;
5      }

```

6 shows a class named `Constants` that has two public constants.

7 Even though constants are considered static members, a constant declaration neither requires nor allows the
8 modifier `static`. Constants can be accessed through the class, as in

```

9      using System;
10     class Test
11     {
12         static void Main() {
13             Console.WriteLine("{0}, {1}", Constants.A, Constants.B);
14         }
15     }

```

16 which prints out the values of `Constants.A` and `Constants.B`, respectively.

17 8.7.2 Fields

18 A *field* is a member that represents a variable associated with an object or class. The example

```

19     class Color
20     {
21         internal ushort redPart;
22         internal ushort bluePart;
23         internal ushort greenPart;
24
25         public Color(ushort red, ushort blue, ushort green) {
26             redPart = red;
27             bluePart = blue;
28             greenPart = green;
29         }
30
31         public static Color Red = new Color(0xFF, 0, 0);
32         public static Color Blue = new Color(0, 0xFF, 0);
33         public static Color Green = new Color(0, 0, 0xFF);
34         public static Color White = new Color(0xFF, 0xFF, 0xFF);
35     }

```

34 shows a `Color` class that has internal instance fields named `redPart`, `bluePart`, and `greenPart`, and
35 static fields named `Red`, `Blue`, `Green`, and `White`

36 The use of static fields in this manner is not ideal. The fields are initialized at some point before they are
37 used, but after this initialization there is nothing to stop a client from changing them. Such a modification
38 could cause unpredictable errors in other programs that use `Color` and assume that the values do not
39 change. **Readonly fields** can be used to prevent such problems. Assignments to a readonly field can only
40 occur as part of the declaration, or in an instance constructor or static constructor in the same class. A static
41 readonly field can be assigned in a static constructor, and a non-static readonly field can be assigned in an
42 instance constructor. Thus, the `Color` class can be enhanced by adding the modifier `readonly` to the static
43 fields:

```

44     class Color
45     {
46         internal ushort redPart;
47         internal ushort bluePart;
48         internal ushort greenPart;
49
50         public Color(ushort red, ushort blue, ushort green) {
51             redPart = red;
52             bluePart = blue;
53             greenPart = green;
54         }
55
56         public static readonly Color Red = new Color(0xFF, 0, 0);
57         public static readonly Color Blue = new Color(0, 0xFF, 0);
58         public static readonly Color Green = new Color(0, 0, 0xFF);
59         public static readonly Color White = new Color(0xFF, 0xFF, 0xFF);
60     }

```

```

1      public static readonly Color Red = new Color(0xFF, 0, 0);
2      public static readonly Color Blue = new Color(0, 0xFF, 0);
3      public static readonly Color Green = new Color(0, 0, 0xFF);
4      public static readonly Color White = new Color(0xFF, 0xFF, 0xFF);
5  }

```

8.7.3 Methods

A **method** is a member that implements a computation or action that can be performed by an object or class. Methods have a (possibly empty) list of formal parameters, a return value (unless the method's *return-type* is **void**), and are either static or non-static. **Static methods** are accessed through the class. **Non-static methods**, which are also called **instance methods**, are accessed through instances of the class. The example

```

11     using System;
12     public class Stack
13     {
14         public static Stack Clone(Stack s) {...}
15         public static Stack Flip(Stack s) {...}
16         public object Pop() {...}
17         public void Push(object o) {...}
18         public override string ToString() {...}
19         ...
20     }
21     class Test
22     {
23         static void Main() {
24             Stack s = new Stack();
25             for (int i = 1; i < 10; i++)
26                 s.Push(i);
27
28             Stack flipped = Stack.Flip(s);
29             Stack cloned = Stack.Clone(s);
30
31             Console.WriteLine("Original stack: " + s.ToString());
32             Console.WriteLine("Flipped stack: " + flipped.ToString());
33             Console.WriteLine("Cloned stack: " + cloned.ToString());
34         }
35     }

```

shows a **Stack** that has several static methods (**Clone** and **Flip**) and several instance methods (**Pop**, **Push**, and **ToString**).

Methods can be overloaded, which means that multiple methods may have the same name so long as they have unique signatures. The signature of a method consists of the name of the method and the number, modifiers, and types of its formal parameters. The signature of a method does not include the return type. The example

```

40     using System;
41     class Test
42     {
43         static void F() {
44             Console.WriteLine("F()");
45         }
46         static void F(object o) {
47             Console.WriteLine("F(object)");
48         }
49         static void F(int value) {
50             Console.WriteLine("F(int)");
51         }
52         static void F(ref int value) {
53             Console.WriteLine("F(ref int)");
54         }
55     }

```



```

1      static void F(int a, int b) {
2          Console.WriteLine("F(int, int)");
3      }
4      static void F(int[] values) {
5          Console.WriteLine("F(int[])");
6      }
7      static void Main() {
8          F();
9          F(1);
10         int i = 10;
11         F(ref i);
12         F((object)1);
13         F(1, 2);
14         F(new int[] {1, 2, 3});
15     }
16 }

```

17 shows a class with a number of methods called F. The output produced is

```

18 F()
19 F(int)
20 F(ref int)
21 F(object)
22 F(int, int)
23 F(int[])

```

24 8.7.4 Properties

25 A **property** is a member that provides access to a characteristic of an object or a class. Examples of
 26 properties include the length of a string, the size of a font, the caption of a window, the name of a customer,
 27 and so on. Properties are a natural extension of fields. Both are named members with associated types, and
 28 the syntax for accessing fields and properties is the same. However, unlike fields, properties do not denote
 29 storage locations. Instead, properties have accessors that specify the statements to be executed when their
 30 values are read or written.

31 Properties are defined with property declarations. The first part of a property declaration looks quite similar
 32 to a field declaration. The second part includes a get accessor and/or a set accessor. In the example below,
 33 the **Button** class defines a **Caption** property.

```

34     public class Button
35     {
36         private string caption;
37         public string Caption {
38             get {
39                 return caption;
40             }
41             set {
42                 caption = value;
43                 Repaint();
44             }
45         }
46     }

```

47 Properties that can be both read and written, such as **Caption**, include both get and set accessors. The get
 48 accessor is called when the property's value is read; the set accessor is called when the property's value is
 49 written. In a set accessor, the new value for the property is made available via an implicit parameter named
 50 **value**.

51 The declaration of properties is relatively straightforward, but the real value of properties is seen when they
 52 are used. For example, the **Caption** property can be read and written in the same way that fields can be read
 53 and written:

```

1      Button b = new Button();
2      b.Caption = "ABC";           // set; causes repaint
3      string s = b.Caption;       // get
4      b.Caption += "DEF";         // get & set; causes repaint

```

8.7.5 Events

An **event** is a member that enables an object or class to provide notifications. A class defines an event by providing an event declaration (which resembles a field declaration, though with an added **event** keyword) and an optional set of event accessors. The type of this declaration must be a delegate type.

An instance of a delegate type encapsulates one or more callable entities. For instance methods, a callable entity consists of an instance and a method on that instance. For static methods, a callable entity consists of just a method. Given a delegate instance and an appropriate set of arguments, one can invoke all of that delegate instance's methods with that set of arguments.

In the example

```

14      public delegate void EventHandler(object sender, EventArgs e);
15      public class Button
16      {
17          public event EventHandler Click;
18          public void Reset() {
19              Click = null;
20          }
21      }

```

the **Button** class defines a **Click** event of type **EventHandler**. Inside the **Button** class, the **Click** member is exactly like a private field of type **EventHandler**. However, outside the **Button** class, the **Click** member can only be used on the left-hand side of the **+=** and **-=** operators. The **+=** operator adds a handler for the event, and the **-=** operator removes a handler for the event. The example

```

26      using System;
27      public class Form1
28      {
29          public Form1() {
30              // Add Button1_Click as an event handler for Button1's Click event
31              Button1.Click += new EventHandler(Button1_Click);
32          }
33          Button Button1 = new Button();
34          void Button1_Click(object sender, EventArgs e) {
35              Console.WriteLine("Button1 was clicked!");
36          }
37          public void Disconnect() {
38              Button1.Click -= new EventHandler(Button1_Click);
39          }
40      }

```

shows a **Form1** class that adds **Button1_Click** as an event handler for **Button1**'s **Click** event. In the **Disconnect** method, that event handler is removed.

For a simple event declaration such as

```

44      public event EventHandler Click;

```

the compiler automatically provides the implementation underlying the **+=** and **-=** operators.

An implementer who wants more control can get it by explicitly providing add and remove accessors. For example, the **Button** class could be rewritten as follows:

```

48      public class Button
49      {
50          private EventHandler handler;
51          public event EventHandler Click {
52              add { handler += value; }
53          }

```

```

1         remove { handler -= value; }
2     }
3 }

```

4 This change has no effect on client code, but allows the `Button` class more implementation flexibility. For
5 example, the event handler for `Click` need not be represented by a field.

6 8.7.6 Operators

7 An **operator** is a member that defines the meaning of an expression operator that can be applied to instances
8 of the class. There are three kinds of operators that can be defined: unary operators, binary operators, and
9 conversion operators.

10 The following example defines a `Digit` type that represents decimal digits—integral values between 0
11 and 9.

```

12     using System;
13     public struct Digit
14     {
15         byte value;
16         public Digit(byte value) {
17             if (value < 0 || value > 9) throw new ArgumentException();
18             this.value = value;
19         }
20         public Digit(int value): this((byte) value) {}
21         public static implicit operator byte(Digit d) {
22             return d.value;
23         }
24         public static explicit operator Digit(byte b) {
25             return new Digit(b);
26         }
27         public static Digit operator+(Digit a, Digit b) {
28             return new Digit(a.value + b.value);
29         }
30         public static Digit operator-(Digit a, Digit b) {
31             return new Digit(a.value - b.value);
32         }
33         public static bool operator==(Digit a, Digit b) {
34             return a.value == b.value;
35         }
36         public static bool operator!=(Digit a, Digit b) {
37             return a.value != b.value;
38         }
39         public override bool Equals(object value) {
40             if (value == null) return false;
41             if (GetType() == value.GetType()) return this == (Digit)value;
42             return false; }
43         public override int GetHashCode() {
44             return value.GetHashCode();
45         }
46         public override string ToString() {
47             return value.ToString();
48         }
49     }

```

```

1      class Test
2      {
3          static void Main() {
4              Digit a = (Digit) 5;
5              Digit b = (Digit) 3;
6              Digit plus = a + b;
7              Digit minus = a - b;
8              bool equals = (a == b);
9              Console.WriteLine("{0} + {1} = {2}", a, b, plus);
10             Console.WriteLine("{0} - {1} = {2}", a, b, minus);
11             Console.WriteLine("{0} == {1} = {2}", a, b, equals);
12         }
13     }

```

14 The `Digit` type defines the following operators:

- 15 • An implicit conversion operator from `Digit` to `byte`.
- 16 • An explicit conversion operator from `byte` to `Digit`.
- 17 • An addition operator that adds two `Digit` values and returns a `Digit` value.
- 18 • A subtraction operator that subtracts one `Digit` value from another, and returns a `Digit` value.
- 19 • The equality (`==`) and inequality (`!=`) operators, which compare two `Digit` values.

20 8.7.7 Indexers

21 An ***indexer*** is a member that enables an object to be indexed in the same way as an array. Whereas
 22 properties enable field-like access, indexers enable array-like access.

23 As an example, consider the `Stack` class presented earlier. The designer of this class might want to expose
 24 array-like access so that it is possible to inspect or alter the items on the stack without performing
 25 unnecessary `Push` and `Pop` operations. That is, class `Stack` is implemented as a linked list, but it also
 26 provides the convenience of array access.

27 Indexer declarations are similar to property declarations, with the main differences being that indexers are
 28 nameless (the “name” used in the declaration is `this`, since `this` is being indexed) and that indexers
 29 include indexing parameters. The indexing parameters are provided between square brackets. The example

```

30     using System;
31     public class Stack
32     {
33         private Node GetNode(int index) {
34             Node temp = first;
35             while (index > 0) {
36                 temp = temp.Next;
37                 index--;
38             }
39             return temp;
40         }
41         public object this[int index] {
42             get {
43                 if (!ValidIndex(index))
44                     throw new Exception("Index out of range.");
45                 else
46                     return GetNode(index).value;
47             }
48             set {
49                 if (!ValidIndex(index))
50                     throw new Exception("Index out of range.");
51                 else
52                     GetNode(index).value = value;
53             }
54         }
55         ...
56     }

```

```

1      class Test
2      {
3          static void Main() {
4              Stack s = new Stack();
5
6              s.Push(1);
7              s.Push(2);
8              s.Push(3);
9
10             s[0] = 33; // Changes the top item from 3 to 33
11             s[1] = 22; // Changes the middle item from 2 to 22
12             s[2] = 11; // Changes the bottom item from 1 to 11
13         }
14     }

```

shows an indexer for the `Stack` class.

8.7.8 Instance constructors

An **instance constructor** is a member that implements the actions required to initialize an instance of a class.

The example

```

17     using System;
18     class Point
19     {
20         public double x, y;
21         public Point() {
22             this.x = 0;
23             this.y = 0;
24         }
25         public Point(double x, double y) {
26             this.x = x;
27             this.y = y;
28         }
29         public static double Distance(Point a, Point b) {
30             double xdiff = a.x - b.x;
31             double ydiff = a.y - b.y;
32             return Math.Sqrt(xdiff * xdiff + ydiff * ydiff);
33         }
34         public override string ToString() {
35             return string.Format("{0}, {1}", x, y);
36         }
37     }
38     class Test
39     {
40         static void Main() {
41             Point a = new Point();
42             Point b = new Point(3, 4);
43             double d = Point.Distance(a, b);
44             Console.WriteLine("Distance from {0} to {1} is {2}", a, b, d);
45         }
46     }

```

shows a `Point` class that provides two public instance constructors, one of which takes no arguments, while the other takes two `double` arguments.

If no instance constructor is supplied for a class, then an empty one with no parameters is automatically provided.

8.7.9 Destructors

A **destructor** is a member that implements the actions required to destruct an instance of a class. Destructors cannot have parameters, they cannot have accessibility modifiers, and they cannot be called explicitly. The destructor for an instance is called automatically during garbage collection.

The example

```

using System;
class Point
{
    public double x, y;
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
    ~Point() {
        Console.WriteLine("Destructed {0}", this);
    }
    public override string ToString() {
        return string.Format("{0}, {1}", x, y);
    }
}

```

shows a `Point` class with a destructor.

8.7.10 Static constructors

A **static constructor** is a member that implements the actions required to initialize a class. Static constructors cannot have parameters, they cannot have accessibility modifiers, and they cannot be called explicitly. The static constructor for a class is called automatically.

The example

```

using Personnel.Data;
class Employee
{
    private static DataSet ds;
    static Employee() {
        ds = new DataSet(...);
    }
    public string Name;
    public decimal Salary;
    ...
}

```

shows an `Employee` class with a static constructor that initializes a static field.

8.7.11 Inheritance

Classes support single inheritance, and the type `object` is the ultimate base class for all classes.

The classes shown in earlier examples all implicitly derive from `object`. The example

```

using System;
class A
{
    public void F() { Console.WriteLine("A.F"); }
}

```

shows a class `A` that implicitly derives from `object`. The example

```

class B: A
{
    public void G() { Console.WriteLine("B.G"); }
}

class Test
{
    static void Main() {
        B b = new B();
        b.F();           // Inherited from A
        b.G();           // Introduced in B
    }
}

```

```

1      A a = b;          // Treat a B as an A
2      a.F();
3  }
4  }

```

5 shows a class **B** that derives from **A**. The class **B** inherits **A**'s **F** method, and introduces a **G** method of its own. Methods, properties, and indexers can be **virtual**, which means that their implementation can be overridden in derived classes. The example

```

8      using System;
9      class A
10     {
11         public virtual void F() { Console.WriteLine("A.F"); }
12     }
13     class B: A
14     {
15         public override void F() {
16             base.F();
17             Console.WriteLine("B.F");
18         }
19     }
20     class Test
21     {
22         static void Main() {
23             B b = new B();
24             b.F();
25             A a = b;
26             a.F();
27         }
28     }

```

29 shows a class **A** with a virtual method **F**, and a class **B** that overrides **F**. The overriding method in **B** contains a call, **base.F()**, which calls the overridden method in **A**.

31 A class can indicate that it is incomplete, and is intended only as a base class for other classes, by including the modifier **abstract**. Such a class is called an **abstract class**. An abstract class can specify **abstract members**—members that a non-abstract derived class must implement. The example

```

34     using System;
35     abstract class A
36     {
37         public abstract void F();
38     }
39     class B: A
40     {
41         public override void F() { Console.WriteLine("B.F"); }
42     }
43     class Test
44     {
45         static void Main() {
46             B b = new B();
47             b.F();
48             A a = b;
49             a.F();
50         }
51     }

```

52 introduces an abstract method **F** in the abstract class **A**. The non-abstract class **B** provides an implementation for this method.

54 8.8 Structs

55 The list of similarities between classes and structs is long—structs can implement interfaces, and can have the same kinds of members as classes. Structs differ from classes in several important ways, however:

structs are value types rather than reference types, and inheritance is not supported for structs. Struct values are stored “on the stack” or “in-line”. Careful programmers can sometimes enhance performance through judicious use of structs.

For example, the use of a struct rather than a class for a `Point` can make a large difference in the number of memory allocations performed at run time. The program below creates and initializes an array of 100 points. With `Point` implemented as a class, 101 separate objects are instantiated—one for the array and one each for the 100 elements.

```

class Point
{
    public int x, y;
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

class Test
{
    static void Main() {
        Point[] points = new Point[100];
        for (int i = 0; i < 100; i++)
            points[i] = new Point(i, i*i);
    }
}

```

If `Point` is instead implemented as a struct, as in

```

struct Point
{
    public int x, y;
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

```

only one object is instantiated—the one for the array. The `Point` instances are allocated in-line within the array. This optimization can be misused. Using structs instead of classes can also make an application run slower or take up more memory, as passing a struct instance by value causes a copy of that struct to be created.

8.9 Interfaces

An interface defines a contract. A class or struct that implements an interface must adhere to its contract. Interfaces can contain methods, properties, events, and indexers as members.

The example

```

interface IExample
{
    string this[int index] { get; set; }
    event EventHandler E;
    void F(int value);
    string P { get; set; }
}

public delegate void EventHandler(object sender, EventArgs e);

```

shows an interface that contains an indexer, an event `E`, a method `F`, and a property `P`.

Interfaces may employ multiple inheritance. In the example

```

interface IControl
{
    void Paint();
}

```



```

1      interface ITextBox: IControl
2      {
3          void SetText(string text);
4      }
5      interface IListBox: IControl
6      {
7          void SetItems(string[] items);
8      }
9      interface IComboBox: ITextBox, IListBox {}

```

the interface `IComboBox` inherits from both `ITextBox` and `IListBox`.

Classes and structs can implement multiple interfaces. In the example

```

12     interface IDataBound
13     {
14         void Bind(Binder b);
15     }
16     public class EditBox: Control, IControl, IDataBound
17     {
18         public void Paint() {...}
19         public void Bind(Binder b) {...}
20     }

```

the class `EditBox` derives from the class `Control` and implements both `IControl` and `IDataBound`.

In the previous example, the `Paint` method from the `IControl` interface and the `Bind` method from `IDataBound` interface are implemented using public members on the `EditBox` class. C# provides an alternative way of implementing these methods that allows the implementing class to avoid having these members be public. Interface members can be implemented using a qualified name. For example, the `EditBox` class could instead be implemented by providing `IControl.Paint` and `IDataBound.Bind` methods.

```

28     public class EditBox: IControl, IDataBound
29     {
30         void IControl.Paint() {...}
31         void IDataBound.Bind(Binder b) {...}
32     }

```

Interface members implemented in this way are called **explicit interface members** because each member explicitly designates the interface member being implemented. Explicit interface members can only be called via the interface. For example, the `EditBox`'s implementation of the `Paint` method can be called only by casting to the `IControl` interface.

```

37     class Test
38     {
39         static void Main() {
40             EditBox editbox = new EditBox();
41             editbox.Paint(); // error: no such method
42             IControl control = editbox;
43             control.Paint(); // calls EditBox's Paint implementation
44         }
45     }

```

8.10 Delegates

Delegates enable scenarios that some other languages have addressed with function pointers. However, unlike function pointers, delegates are object-oriented and type-safe.

A delegate declaration defines a class that is derived from the class `System.Delegate`. A delegate instance encapsulates one or more methods, each of which is referred to as a **callable entity**. For instance methods, a callable entity consists of an instance and a method on that instance. For static methods, a callable entity consists of just a method. Given a delegate instance and an appropriate set of arguments, one can invoke all of that delegate instance's methods with that set of arguments.

An interesting and useful property of a delegate instance is that it does not know or care about the classes of the methods it encapsulates; all that matters is that those methods be compatible (§22.1) with the delegate's type. This makes delegates perfectly suited for “anonymous” invocation. This is a powerful capability.

There are three steps in defining and using delegates: declaration, instantiation, and invocation. Delegates are declared using delegate declaration syntax. The example

```
delegate void SimpleDelegate();
```

declares a delegate named `SimpleDelegate` that takes no arguments and returns no result.

The example

```
class Test
{
    static void F() {
        System.Console.WriteLine("Test.F");
    }

    static void Main() {
        SimpleDelegate d = new SimpleDelegate(F);
        d();
    }
}
```

creates a `SimpleDelegate` instance and then immediately calls it.

There is not much point in instantiating a delegate for a method and then immediately calling that method via the delegate, as it would be simpler to call the method directly. Delegates really show their usefulness when their anonymity is used. The example

```
void MultiCall(SimpleDelegate d, int count) {
    for (int i = 0; i < count; i++)
        d();
}
```

shows a `MultiCall` method that repeatedly calls a `SimpleDelegate`. The `MultiCall` method doesn't know or care about the type of the target method for the `SimpleDelegate`, what accessibility that method has, or whether or not that method is static. All that matters is that the target method is compatible (§22.1) with `SimpleDelegate`.

8.11 Enums

An enum type declaration defines a type name for a related group of symbolic constants. Enums are used for “multiple choice” scenarios, in which a runtime decision is made from a fixed number of choices that are known at compile-time.

The example

```
enum Color
{
    Red,
    Blue,
    Green
}

class Shape
{
    public void Fill(Color color) {
        switch(color) {
            case Color.Red:
                ...
                break;
            case Color.Blue:
                ...
                break;
        }
    }
}
```

```

1         case Color.Green:
2             ...
3             break;
4         default:
5             break;
6     }
7 }
8

```

shows a `Color` enum and a method that uses this enum. The signature of the `Fill` method makes it clear that the shape can be filled with one of the given colors.

The use of enums is superior to the use of integer constants—as is common in languages without enums—because the use of enums makes the code more readable and self-documenting. The self-documenting nature of the code also makes it possible for the development tool to assist with code writing and other “designer” activities. For example, the use of `Color` rather than `int` for a parameter type enables smart code editors to suggest `Color` values.

8.12 Namespaces and assemblies

The programs presented so far have stood on their own except for dependence on a few system-provided classes such as `System.Console`. It is far more common, however, for real-world applications to consist of several different pieces, each compiled separately. For example, a corporate application might depend on several different components, including some developed internally and some purchased from independent software vendors.

Namespaces and **assemblies** enable this component-based system. Namespaces provide a logical organizational system. Namespaces are used both as an “internal” organization system for a program, and as an “external” organization system—a way of presenting program elements that are exposed to other programs.

Assemblies are used for physical packaging and deployment. An assembly may contain types, the executable code used to implement these types, and references to other assemblies.

To demonstrate the use of namespaces and assemblies, this section revisits the “hello, world” program presented earlier, and splits it into two pieces: a class library that provides messages and a console application that displays them.

The class library will contain a single class named `HelloMessage`. The example

```

32 // HelloLibrary.cs
33 namespace CSharp.Introduction
34 {
35     public class HelloMessage
36     {
37         public string Message {
38             get {
39                 return "hello, world";
40             }
41         }
42     }
43 }

```

shows the `HelloMessage` class in a namespace named `CSharp.Introduction`. The `HelloMessage` class provides a read-only property named `Message`. Namespaces can nest, and the declaration

```

46 namespace CSharp.Introduction
47 {...}

```

is shorthand for two levels of namespace nesting:

```

49 namespace CSharp
50 {
51     namespace Introduction
52     {...}
53 }

```

The next step in the componentization of “hello, world” is to write a console application that uses the `HelloMessage` class. The fully qualified name for the class—`CSharp.Introduction>HelloMessage`—could be used, but this name is quite long and unwieldy. An easier way is to use a ***using namespace directive***, which makes it possible to use all of the types in a namespace without qualification. The example

```
// HelloApp.cs
using CSharp.Introduction;
class HelloApp
{
    static void Main() {
        HelloMessage m = new HelloMessage();
        System.Console.WriteLine(m.Message);
    }
}
```

shows a `using namespace` directive that refers to the `CSharp.Introduction` namespace. The occurrences of `HelloMessage` are shorthand for `CSharp.Introduction>HelloMessage`.

C# also enables the definition and use of aliases. A ***using alias directive*** defines an alias for a type. Such aliases can be useful in situation in which name collisions occur between two class libraries, or when a small number of types from a much larger namespace are being used. The example

```
using MessageSource = CSharp.Introduction>HelloMessage;
```

shows a `using alias` directive that defines `MessageSource` as an alias for the `HelloMessage` class.

The code we have written can be compiled into a class library containing the class `HelloMessage` and an application containing the class `HelloApp`. The details of this compilation step might differ based on the compiler or tool being used. A command-line compiler might enable compilation of a class library and an application that uses that library with the following command-line invocations:

```
csc /target:library HelloLibrary.cs
csc /reference:HelloLibrary.dll HelloApp.cs
```

which produce a class library named `HelloLibrary.dll` and an application named `HelloApp.exe`.

8.13 Versioning

Versioning is the process of evolving a component over time in a compatible manner. A new version of a component is ***source compatible*** with a previous version if code that depends on the previous version can, when recompiled, work with the new version. In contrast, a new version of a component is ***binary compatible*** if an application that depended on the old version can, without recompilation, work with the new version.

Most languages do not support binary compatibility at all, and many do little to facilitate source compatibility. In fact, some languages contain flaws that make it impossible, in general, to evolve a class over time without breaking at least some client code.

As an example, consider the situation of a base class author who ships a class named `Base`. In the first version, `Base` contains no method `F`. A component named `Derived` derives from `Base`, and introduces an `F`. This `Derived` class, along with the class `Base` on which it depends, is released to customers, who deploy to numerous clients and servers.

```
// Author A
namespace A
{
    public class Base    // version 1
    {
    }
}
```

```

1      // Author B
2      namespace B
3      {
4          class Derived: A.Base
5          {
6              public virtual void F() {
7                  System.Console.WriteLine("Derived.F");
8              }
9          }
10     }

```

11 So far, so good, but now the versioning trouble begins. The author of **Base** produces a new version, giving it
 12 its own method **F**.

```

13     // Author A
14     namespace A
15     {
16         public class Base    // version 2
17         {
18             public virtual void F() {    // added in version 2
19                 System.Console.WriteLine("Base.F");
20             }
21         }
22     }

```

23 This new version of **Base** should be both source and binary compatible with the initial version. (If it weren't
 24 possible to simply add a method then a base class could never evolve.) Unfortunately, the new **F** in **Base**
 25 makes the meaning of **Derived**'s **F** unclear. Did **Derived** mean to override **Base**'s **F**? This seems unlikely,
 26 since when **Derived** was compiled, **Base** did not even have an **F**! Further, if **Derived**'s **F** does override
 27 **Base**'s **F**, then it must adhere to the contract specified by **Base**—a contract that was unspecified when
 28 **Derived** was written. In some cases, this is impossible. For example, **Base**'s **F** might require that overrides
 29 of it always call the base. **Derived**'s **F** could not possibly adhere to such a contract.

30 C# addresses this versioning problem by requiring developers to state their intent clearly. In the original
 31 code example, the code was clear, since **Base** did not even have an **F**. Clearly, **Derived**'s **F** is intended as a
 32 new method rather than an override of a base method, since no base method named **F** exists.

33 If **Base** adds an **F** and ships a new version, then the intent of a binary version of **Derived** is still clear—
 34 **Derived**'s **F** is semantically unrelated, and should not be treated as an override.

35 However, when **Derived** is recompiled, the meaning is unclear—the author of **Derived** may intend its **F** to
 36 override **Base**'s **F**, or to hide it. Since the intent is unclear, the compiler produces a warning, and by default
 37 makes **Derived**'s **F** hide **Base**'s **F**. This course of action duplicates the semantics for the case in which
 38 **Derived** is not recompiled. The warning that is generated alerts **Derived**'s author to the presence of the
 39 **F** method in **Base**.

40 If **Derived**'s **F** is semantically unrelated to **Base**'s **F**, then **Derived**'s author can express this intent—and,
 41 in effect, turn off the warning—by using the **new** keyword in the declaration of **F**.

```

42     // Author A
43     namespace A
44     {
45         public class Base    // version 2
46         {
47             public virtual void F() { // added in version 2
48                 System.Console.WriteLine("Base.F");
49             }
50         }
51     }

```

```

1      // Author B
2      namespace B
3      {
4          class Derived: A.Base    // version 2a: new
5          {
6              new public virtual void F() {
7                  System.Console.WriteLine("Derived.F");
8              }
9          }
10     }

```

On the other hand, `Derived`'s author might investigate further, and decide that `Derived`'s `F` should override `Base`'s `F`. This intent can be specified by using the `override` keyword, as shown below.

```

13     // Author A
14     namespace A
15     {
16         public class Base          // version 2
17         {
18             public virtual void F() { // added in version 2
19                 System.Console.WriteLine("Base.F");
20             }
21         }
22     }
23
24     // Author B
25     namespace B
26     {
27         class Derived: A.Base    // version 2b: override
28         {
29             public override void F() {
30                 base.F();
31                 System.Console.WriteLine("Derived.F");
32             }
33         }
34     }

```

The author of `Derived` has one other option, and that is to change the name of `F`, thus completely avoiding the name collision. Although this change would break source and binary compatibility for `Derived`, the importance of this compatibility varies depending on the scenario. If `Derived` is not exposed to other programs, then changing the name of `F` is likely a good idea, as it would improve the readability of the program—there would no longer be any confusion about the meaning of `F`.

8.14 Attributes

C# is an imperative language, but like all imperative languages it does have some declarative elements. For example, the accessibility of a method in a class is specified by declaring it `public`, `protected`, `internal`, `protected internal`, or `private`. C# generalizes this capability, so that programmers can invent new kinds of declarative information, attach this declarative information to various program entities, and retrieve this declarative information at run-time. Programs specify this additional declarative information by defining and using attributes (§24).

For instance, a framework might define a `HelpAttribute` attribute that can be placed on program elements such as classes and methods, enabling developers to provide a mapping from program elements to documentation for them. The example

```

49     using System;
50     [AttributeUsage(AttributeTargets.All)]
51     public class HelpAttribute: Attribute
52     {
53         public HelpAttribute(string url) {
54             this.url = url;
55         }
56
57         public string Topic = null;
58         private string url;

```

```

1      public string Url {
2          get { return url; }
3      }
4  }

```

5 defines an attribute class named `HelpAttribute`, or `Help` for short, that has one positional parameter
6 (`string url`) and one named parameter (`string Topic`). Positional parameters are defined by the
7 formal parameters for public instance constructors of the attribute class, and named parameters are defined
8 by public non-static read-write fields and properties of the attribute class.

9 The example

```

10      [Help("http://www.mycompany.com/.../Class1.htm")]
11      public class Class1
12      {
13          [Help("http://www.mycompany.com/.../Class1.htm", Topic = "F")]
14          public void F() {}
15      }

```

16 shows several uses of the attribute `Help`.

17 Attribute information for a given program element can be retrieved at run-time by using reflection support.

18 The example

```

19      using System;
20      class Test
21      {
22          static void Main() {
23              Type type = typeof(Class1);
24              object[] arr = type.GetCustomAttributes(typeof(HelpAttribute),
25 true);
26              if (arr.Length == 0)
27                  Console.WriteLine("Class1 has no Help attribute.");
28              else {
29                  HelpAttribute ha = (HelpAttribute) arr[0];
30                  Console.WriteLine("Url = {0}, Topic = {1}", ha.Url, ha.Topic);
31              }
32          }
33      }

```

34 checks to see if `Class1` has a `Help` attribute, and writes out the associated `Topic` and `Url` values if the
35 attribute is present.

36 **End of informative text.**

37

9. Lexical structure

9.1 Programs

A C# **program** consists of one or more source files, known formally as **compilation units** (§16.1). A source file is an ordered sequence of Unicode characters. Source files typically have a one-to-one correspondence with files in a file system, but this correspondence is not required.

Conceptually speaking, a program is compiled using three steps:

1. Transformation, which converts a file from a particular character repertoire and encoding scheme into a sequence of Unicode characters.
2. Lexical analysis, which translates a stream of Unicode input characters into a stream of tokens.
3. Syntactic analysis, which translates the stream of tokens into executable code.

Conforming implementations must accept Unicode source files encoded with the UTF-8 encoding form (as defined by the Unicode standard), and transform them into a sequence of Unicode characters.

Implementations may choose to accept and transform additional character encoding schemes (such as UTF-16, UTF-32, or non-Unicode character mappings).

[*Note:* It is beyond the scope of this standard to define how a file using a character representation other than Unicode might be transformed into a sequence of Unicode characters. During such transformation, however, it is recommended that the usual line-separating character (or sequence) in the other character set be translated to the two-character sequence consisting of the Unicode carriage-return character followed by Unicode line-feed character. For the most part this transformation will have no visible effects; however, it will affect the interpretation of verbatim string literal tokens (§9.4.4.5). The purpose of this recommendation is to allow a verbatim string literal to produce the same character sequence when its source file is moved between systems that support differing non-Unicode character sets, in particular, those using differing character sequences for line-separation. *end note*]

9.2 Grammars

This specification presents the syntax of the C# programming language using two grammars. The **lexical grammar** (§9.2.1) defines how Unicode characters are combined to form line terminators, white space, comments, tokens, and pre-processing directives. The **syntactic grammar** (§9.2.2) defines how the tokens resulting from the lexical grammar are combined to form C# programs.

9.2.1 Lexical grammar

The lexical grammar of C# is presented in §9.3, §9.4, and §9.5. The terminal symbols of the lexical grammar are the characters of the Unicode character set, and the lexical grammar specifies how characters are combined to form tokens (§9.4), white space (§9.3.3), comments (§9.3.2), and pre-processing directives (§9.5).

Every source file in a C# program must conform to the *input* production of the lexical grammar (§9.3).

9.2.2 Syntactic grammar

The syntactic grammar of C# is presented in the chapters and appendices that follow this chapter. The terminal symbols of the syntactic grammar are the tokens defined by the lexical grammar, and the syntactic grammar specifies how tokens are combined to form C# programs.

Every source file in a C# program must conform to the *compilation-unit* production (§16.1) of the syntactic grammar.

9.3 Lexical analysis

The *input* production defines the lexical structure of a C# source file. Each source file in a C# program must conform to this lexical grammar production.

```

input::
    input-sectionopt

input-section::
    input-section-part
    input-section input-section-part

input-section-part::
    input-elementsopt new-line
    pp-directive

input-elements::
    input-element
    input-elements input-element

input-element::
    whitespace
    comment
    token

```

Five basic elements make up the lexical structure of a C# source file: Line terminators (§9.3.1), white space (§9.3.3), comments (§9.3.2), tokens (§9.4), and pre-processing directives (§9.5). Of these basic elements, only tokens are significant in the syntactic grammar of a C# program (§9.2.2).

The lexical processing of a C# source file consists of reducing the file into a sequence of tokens which becomes the input to the syntactic analysis. Line terminators, white space, and comments can serve to separate tokens, and pre-processing directives can cause sections of the source file to be skipped, but otherwise these lexical elements have no impact on the syntactic structure of a C# program.

When several lexical grammar productions match a sequence of characters in a source file, the lexical processing always forms the longest possible lexical element. For example, the character sequence `//` is processed as the beginning of a single-line comment because that lexical element is longer than a single / token.

9.3.1 Line terminators

Line terminators divide the characters of a C# source file into lines.

```

new-line::
    Carriage return character (U+000D)
    Line feed character (U+000A)
    Carriage return character (U+000D) followed by line feed character (U+000A)
    Line separator character (U+2028)
    Paragraph separator character (U+2029)

```

For compatibility with source code editing tools that add end-of-file markers, and to enable a source file to be viewed as a sequence of properly terminated lines, the following transformations are applied, in order, to every source file in a C# program:

- If the last character of the source file is a Control-Z character (U+001A), this character is deleted.
- A carriage-return character (U+000D) is added to the end of the source file if that source file is non-empty and if the last character of the source file is not a carriage return (U+000D), a line feed (U+000A), a line separator (U+2028), or a paragraph separator (U+2029).

9.3.2 Comments

Two forms of comments are supported: delimited comments and single-line comments.

A **delimited comment** begins with the characters `/*` and ends with the characters `*/`. Delimited comments can occupy a portion of a line, a single line, or multiple lines. [Example: The example

```

1  /* Hello, world program
2      This program writes "hello, world" to the console
3  */
4  class Hello
5  {
6      static void Main() {
7          System.Console.WriteLine("hello, world");
8      }
9  }
10
11

```

includes a delimited comment. *end example*]

A **single-line comment** begins with the characters `//` and extends to the end of the line. [Example: The example

```

15 // Hello, world program
16 // This program writes "hello, world" to the console
17 //
18 class Hello // any name will do for this class
19 {
20     static void Main() { // this method must be named "Main"
21         System.Console.WriteLine("hello, world");
22     }
23 }

```

shows several single-line comments. *end example*]

comment::

single-line-comment

delimited-comment

single-line-comment::

// input-characters_{opt}

input-characters::

input-character

input-characters input-character

input-character::

Any Unicode character except a *new-line-character*

new-line-character::

Carriage return character (U+000D)

Line feed character (U+000A)

Line separator character (U+2028)

Paragraph separator character (U+2029)

delimited-comment::

/ delimited-comment-characters_{opt} */*

delimited-comment-characters::

delimited-comment-character

delimited-comment-characters delimited-comment-character

delimited-comment-character::

not-asterisk

** not-slash*

not-asterisk::

Any Unicode character except `*`

not-slash::

Any Unicode character except `/`

Comments do not nest. The character sequences `/*` and `*/` have no special meaning within a single-line comment, and the character sequences `//` and `/*` have no special meaning within a delimited comment.

Comments are not processed within character and string literals.

9.3.3 White space

White space is defined as any character with Unicode class Zs (which includes the space character) as well as the horizontal tab character, the vertical tab character, and the form feed character.

```
whitespace::
    Any character with Unicode class Zs
    Horizontal tab character (U+0009)
    Vertical tab character (U+000B)
    Form feed character (U+000C)
```

9.4 Tokens

There are several kinds of **tokens**: identifiers, keywords, literals, operators, and punctuators. White space and comments are not tokens, though they act as separators for tokens.

```
token::
    identifier
    keyword
    integer-literal
    real-literal
    character-literal
    string-literal
    operator-or-punctuator
```

9.4.1 Unicode escape sequences

A Unicode escape sequence represents a Unicode character. Unicode escape sequences are processed in identifiers (§9.4.2), regular string literals (§9.4.4.5), and character literals (§9.4.4.4). A Unicode character escape is not processed in any other location (for example, to form an operator, punctuator, or keyword).

```
unicode-escape-sequence::
    \u hex-digit hex-digit hex-digit hex-digit
    \U hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit
```

A Unicode escape sequence represents the single Unicode character formed by the hexadecimal number following the `\u` or `\U` characters. Since C# uses a 16-bit encoding of Unicode characters in characters and string values, a Unicode character in the range U+10000 to U+10FFFF is represented using two Unicode surrogate characters. Unicode characters with code points above 0x10FFFF are not supported.

Multiple translations are not performed. For instance, the string literal `"\u005Cu005C"` is equivalent to `"\u005C"` rather than `"\"`. [Note: The Unicode value `\u005C` is the character `"\"`. *end note*]

[Example: The example

```
class Class1
{
    static void Test(bool \u0066) {
        char c = '\u0066';
        if (\u0066)
            System.Console.WriteLine(c.ToString());
    }
}
```

shows several uses of `\u0066`, which is the escape sequence for the letter `"f"`. The program is equivalent to

```

1      class Class1
2      {
3          static void Test(bool f) {
4              char c = 'f';
5              if (f)
6                  System.Console.WriteLine(c.ToString());
7          }
8      }

```

9 *end example]*

10 9.4.2 Identifiers

11 The rules for identifiers rules given in this section correspond exactly to those recommended by the Unicode
 12 Standard Annex 15 except that underscore is allowed as an initial character (as is traditional in the
 13 C programming language), Unicode escape sequences are permitted in identifiers, and the “@” character is
 14 allowed as a prefix to enable keywords to be used as identifiers.

```

15     identifier::
16         available-identifier
17         @ identifier-or-keyword
18
19     available-identifier::
20         An identifier-or-keyword that is not a keyword
21
22     identifier-or-keyword::
23         identifier-start-character identifier-part-charactersopt
24
25     identifier-start-character::
26         letter-character
27         _ (the underscore character U+005F)
28
29     identifier-part-characters::
30         identifier-part-character
31         identifier-part-characters identifier-part-character
32
33     identifier-part-character::
34         letter-character
35         decimal-digit-character
36         connecting-character
37         combining-character
38         formatting-character
39
40     letter-character::
41         A Unicode character of classes Lu, Ll, Lt, Lm, Lo, or Nl
42         A unicode-escape-sequence representing a character of classes Lu, Ll, Lt, Lm, Lo, or Nl
43
44     combining-character::
45         A Unicode character of classes Mn or Mc
46         A unicode-escape-sequence representing a character of classes Mn or Mc
47
48     decimal-digit-character::
49         A Unicode character of the class Nd
50         A unicode-escape-sequence representing a character of the class Nd
51
52     connecting-character::
53         A Unicode character of the class Pc
54         A unicode-escape-sequence representing a character of the class Pc
55
56     formatting-character::
57         A Unicode character of the class Cf
58         A unicode-escape-sequence representing a character of the class Cf

```

[*Note:* For information on the Unicode character classes mentioned above, see *The Unicode Standard, Version 3.0*, §4.5.) *end note*]

[*Example:* Examples of valid identifiers include “`identifier1`”, “`_identifier2`”, and “`@if`”. *end example*]

An identifier in a conforming program must be in the canonical format defined by Unicode Normalization Form C, as defined by Unicode Standard Annex 15. The behavior when encountering an identifier not in Normalization Form C is implementation-defined; however, a diagnostic is not required.

The prefix “`@`” enables the use of keywords as identifiers, which is useful when interfacing with other programming languages. The character `@` is not actually part of the identifier, so the identifier might be seen in other languages as a normal identifier, without the prefix. An identifier with an `@` prefix is called a **verbatim identifier**. [*Note:* Use of the `@` prefix for identifiers that are not keywords is permitted, but strongly discouraged as a matter of style. *end note*]

[*Example:* The example:

```
class @class
{
    public static void @static(bool @bool) {
        if (@bool)
            System.Console.WriteLine("true");
        else
            System.Console.WriteLine("false");
    }
}

class Class1
{
    static void M() {
        cl\u0061ss.st\u0061tic(true);
    }
}
```

defines a class named “`class`” with a static method named “`static`” that takes a parameter named “`bool`”. Note that since Unicode escapes are not permitted in keywords, the token “`cl\u0061ss`” is an identifier, and is the same identifier as “`@class`”. *end example*]

Two identifiers are considered the same if they are identical after the following transformations are applied, in order:

- The prefix “`@`”, if used, is removed.
- Each *unicode-escape-sequence* is transformed into its corresponding Unicode character.
- Any *formatting-characters* are removed.

Identifiers containing two consecutive underscore characters (U+005F) are reserved for use by the implementation; however, no diagnostic is required if such an identifier is defined. [*Note:* For example, an implementation might provide extended keywords that begin with two underscores. *end note*]

9.4.3 Keywords

A **keyword** is an identifier-like sequence of characters that is reserved, and cannot be used as an identifier except when prefaced by the `@` character.

keyword:: one of

abstract	as	base	bool	break
byte	case	catch	char	checked
class	const	continue	decimal	default
delegate	do	double	else	enum
event	explicit	extern	false	finally
fixed	float	for	foreach	goto
if	implicit	in	int	interface
internal	is	lock	long	namespace
new	null	object	operator	out
override	params	private	protected	public
readonly	ref	return	sbyte	sealed
short	sizeof	stackalloc	static	string
struct	switch	this	throw	true
try	typeof	uint	ulong	unchecked
unsafe	ushort	using	virtual	void
volatile	while			

In some places in the grammar, specific identifiers have special meaning, but are not keywords. [Note: For example, within a property declaration, the “get” and “set” identifiers have special meaning (§17.6.2). An identifier other than `get` or `set` is never permitted in these locations, so this use does not conflict with a use of these words as identifiers. *end note*]

9.4.4 Literals

A **literal** is a source code representation of a value.

literal::

boolean-literal
integer-literal
real-literal
character-literal
string-literal
null-literal

9.4.4.1 Boolean literals

There are two boolean literal values: `true` and `false`.

boolean-literal::

`true`
`false`

The type of a *boolean-literal* is `bool`.

9.4.4.2 Integer literals

Integer literals are used to write values of types `int`, `uint`, `long`, and `ulong`. Integer literals have two possible forms: decimal and hexadecimal.

integer-literal::

decimal-integer-literal
hexadecimal-integer-literal

decimal-integer-literal::

decimal-digits *integer-type-suffix*_{opt}

decimal-digits::

decimal-digit
decimal-digits *decimal-digit*

decimal-digit:: one of

0 1 2 3 4 5 6 7 8 9

integer-type-suffix:: one of

U u L l UL Ul uL ul LU Lu lU lu

hexadecimal-integer-literal::

0x *hex-digits* *integer-type-suffix*_{opt}

0X *hex-digits* *integer-type-suffix*_{opt}

hex-digits::

hex-digit

hex-digits *hex-digit*

hex-digit:: one of

0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f

The type of an integer literal is determined as follows:

- If the literal has no suffix, it has the first of these types in which its value can be represented: `int`, `uint`, `long`, `ulong`.
- If the literal is suffixed by `U` or `u`, it has the first of these types in which its value can be represented: `uint`, `ulong`.
- If the literal is suffixed by `L` or `l`, it has the first of these types in which its value can be represented: `long`, `ulong`.
- If the literal is suffixed by `UL`, `Ul`, `uL`, `ul`, `LU`, `Lu`, `lU`, or `lu`, it is of type `ulong`.

If the value represented by an integer literal is outside the range of the `ulong` type, a compile-time error occurs.

[*Note*: As a matter of style, it is suggested that “`L`” be used instead of “`l`” when writing literals of type `long`, since it is easy to confuse the letter “`l`” with the digit “`1`”. *end note*]

To permit the smallest possible `int` and `long` values to be written as decimal integer literals, the following two rules exist:

- When a *decimal-integer-literal* with the value 2147483648 (2^{31}) and no *integer-type-suffix* appears as the token immediately following a unary minus operator token (§14.6.2), the result is a constant of type `int` with the value -2147483648 (-2^{31}). In all other situations, such a *decimal-integer-literal* is of type `uint`.
- When a *decimal-integer-literal* with the value 9223372036854775808 (2^{63}) and no *integer-type-suffix* or the *integer-type-suffix* `L` or `l` appears as the token immediately following a unary minus operator token (§14.6.2), the result is a constant of type `long` with the value -9223372036854775808 (-2^{63}). In all other situations, such a *decimal-integer-literal* is of type `ulong`.

9.4.4.3 Real literals

Real literals are used to write values of types `float`, `double`, and `decimal`.

real-literal::

decimal-digits . *decimal-digits* *exponent-part*_{opt} *real-type-suffix*_{opt}

. *decimal-digits* *exponent-part*_{opt} *real-type-suffix*_{opt}

decimal-digits *exponent-part* *real-type-suffix*_{opt}

decimal-digits *real-type-suffix*

exponent-part::

e *sign*_{opt} *decimal-digits*

E *sign*_{opt} *decimal-digits*

sign:: one of

+ -

real-type-suffix:: one of
F f D d M m

If no *real-type-suffix* is specified, the type of the real literal is `double`. Otherwise, the *real-type-suffix* determines the type of the real literal, as follows:

- A real literal suffixed by F or f is of type `float`. [Example: For example, the literals 1f, 1.5f, 1e10f, and 123.456F are all of type `float`. end example]
- A real literal suffixed by D or d is of type `double`. [Example: For example, the literals 1d, 1.5d, 1e10d, and 123.456D are all of type `double`. end example]
- A real literal suffixed by M or m is of type `decimal`. [Example: For example, the literals 1m, 1.5m, 1e10m, and 123.456M are all of type `decimal`. end example] This literal is converted to a `decimal` value by taking the exact value, and, if necessary, rounding to the nearest representable value using banker's rounding (§11.1.6). Any scale apparent in the literal is preserved unless the value is rounded or the value is zero (in which latter case the sign and scale will be 0). [Note: Hence, the literal 2.900m will be parsed to form the `decimal` with sign 0, coefficient 2900, and scale 3. end note]

If the specified literal cannot be represented in the indicated type, a compile-time error occurs.

The value of a real literal having type `float` or `double` is determined by using the IEEE “round to nearest” mode.

9.4.4.4 Character literals

A character literal represents a single character, and usually consists of a character in quotes, as in 'a'.

character-literal::
' character '

character::
single-character
simple-escape-sequence
hexadecimal-escape-sequence
unicode-escape-sequence

single-character::
Any character except ' (U+0027), \ (U+005C), and *new-line-character*

simple-escape-sequence:: one of
' \" \\ \0 \a \b \f \n \r \t \v

hexadecimal-escape-sequence::
\x hex-digit hex-digit_{opt} hex-digit_{opt} hex-digit_{opt}

[Note: A character that follows a backslash character (\) in a *character* must be one of the following characters: ', ", \, 0, a, b, f, n, r, t, u, U, x, v. Otherwise, a compile-time error occurs. end note]

A hexadecimal escape sequence represents a single Unicode character, with the value formed by the hexadecimal number following “\x”.

If the value represented by a character literal is greater than U+FFFF, a compile-time error occurs.

A Unicode character escape sequence (§9.4.1) in a character literal must be in the range U+0000 to U+FFFF.

A simple escape sequence represents a Unicode character encoding, as described in the table below.

Escape sequence	Character name	Unicode encoding
\'	Single quote	0x0027
\"	Double quote	0x0022
\\	Backslash	0x005C
\0	Null	0x0000
\a	Alert	0x0007
\b	Backspace	0x0008
\f	Form feed	0x000C
\n	New line	0x000A
\r	Carriage return	0x000D
\t	Horizontal tab	0x0009
\v	Vertical tab	0x000B

The type of a *character-literal* is `char`.

9.4.4.5 String literals

C# supports two forms of string literals: ***regular string literals*** and ***verbatim string literals***. A regular string literal consists of zero or more characters enclosed in double quotes, as in `"hello, world"`, and may include both simple escape sequences (such as `\t` for the tab character), and hexadecimal and Unicode escape sequences.

A verbatim string literal consists of an `@` character followed by a double-quote character, zero or more characters, and a closing double-quote character. A simple example is `@"hello, world"`. In a verbatim string literal, the characters between the delimiters are interpreted verbatim, with the only exception being a *quote-escape-sequence*. In particular, simple escape sequences, and hexadecimal and Unicode escape sequences are not processed in verbatim string literals. A verbatim string literal may span multiple lines.

string-literal::

regular-string-literal

verbatim-string-literal

regular-string-literal::

" *regular-string-literal-characters*_{opt} "

regular-string-literal-characters::

regular-string-literal-character

regular-string-literal-characters *regular-string-literal-character*

regular-string-literal-character::

single-regular-string-literal-character

simple-escape-sequence

hexadecimal-escape-sequence

unicode-escape-sequence

single-regular-string-literal-character::

Any character except " (U+0022), \ (U+005C), and *new-line-character*

verbatim-string-literal::

@ " *verbatim-string-literal-characters*_{opt} "

```

1  verbatim-string-literal-characters::
2      verbatim-string-literal-character
3      verbatim-string-literal-characters verbatim-string-literal-character
4
5  verbatim-string-literal-character::
6      single-verbatim-string-literal-character
7      quote-escape-sequence
8
9  single-verbatim-string-literal-character::
10     any character except "
11
12  quote-escape-sequence::
13     ""

```

[*Note:* A character that follows a backslash character (\) in a *regular-string-literal-character* must be one of the following characters: ' , " , \ , 0 , a , b , f , n , r , t , u , U , x , v . Otherwise, a compile-time error occurs. *end note*]

[*Example:* The example

```

15     string a = "Happy birthday, Joel";           // Happy birthday, Joel
16     string b = @"Happy birthday, Joel";          // Happy birthday, Joel
17     string c = "hello \t world";                 // hello      world
18     string d = @"hello \t world";                 // hello \t world
19     string e = "Joe said \"Hello\" to me";        // Joe said "Hello" to me
20     string f = @"Joe said ""Hello"" to me";       // Joe said "Hello" to me
21     string g = "\\server\\share\\file.txt";        // \\server\\share\\file.txt
22     string h = @"\\server\\share\\file.txt";      // \\server\\share\\file.txt
23     string i = "one\r\ntwo\r\nthree";
24     string j = @"one
25     two
26     three";

```

shows a variety of string literals. The last string literal, `j`, is a verbatim string literal that spans multiple lines. The characters between the quotation marks, including white space such as new line characters, are preserved verbatim. *end example*]

[*Note:* Since a hexadecimal escape sequence can have a variable number of hex digits, the string literal `"\x123"` contains a single character with hex value 123. To create a string containing the character with hex value 12 followed by the character 3, one could write `"\x00123"` or `"\x12" + "3"` instead. *end note*]

The type of a *string-literal* is `string`.

Each string literal does not necessarily result in a new string instance. When two or more string literals that are equivalent according to the string equality operator (§14.9.7), appear in the same assembly, these string literals refer to the same string instance. [*Example:* For instance, the output produced by

```

37     class Test
38     {
39         static void Main() {
40             object a = "hello";
41             object b = "hello";
42             System.Console.WriteLine(a == b);
43         }
44     }

```

is `True` because the two literals refer to the same string instance. *end example*]

9.4.4.6 The null literal

```

47     null-literal::
48         null

```

The type of a *null-literal* is the null type.

9.4.5 Operators and punctuators

There are several kinds of operators and punctuators. Operators are used in expressions to describe operations involving one or more operands. [*Example:* For example, the expression `a + b` uses the `+` operator to add the two operands `a` and `b`. *end example*] Punctuators are for grouping and separating.

operator-or-punctuator:: one of

{	}	[	]	(	)	.	,	:	;
+	-	*	/	%	&		^	!	~
=	<	>	?	++	--	&&		<<	>>
==	!=	<=	>=	+=	-=	*=	/=	%=	&=
=	^=	<<=	>>=	->					

9.5 Pre-processing directives

The pre-processing directives provide the ability to conditionally skip sections of source files, to report error and warning conditions, and to delineate distinct regions of source code. [*Note:* The term “pre-processing directives” is used only for consistency with the C and C++ programming languages. In C#, there is no separate pre-processing step; pre-processing directives are processed as part of the lexical analysis phase. *end note*]

pp-directive::

pp-declaration
pp-conditional
pp-line
pp-diagnostic
pp-region

The following pre-processing directives are available:

- `#define` and `#undef`, which are used to define and undefine, respectively, conditional compilation symbols (§9.5.3).
- `#if`, `#elif`, `#else`, and `#endif`, which are used to conditionally skip sections of source code (§9.5.1).
- `#line`, which is used to control line numbers emitted for errors and warnings (§9.5.7).
- `#error` and `#warning`, which are used to issue errors and warnings, respectively (§9.5.5).
- `#region` and `#endregion`, which are used to explicitly mark sections of source code (§9.5.6).

A pre-processing directive always occupies a separate line of source code and always begins with a `#` character and a pre-processing directive name. White space may occur before the `#` character and between the `#` character and the directive name.

A source line containing a `#define`, `#undef`, `#if`, `#elif`, `#else`, `#endif`, or `#line` directive may end with a single-line comment. Delimited comments (the `/* */` style of comments) are not permitted on source lines containing pre-processing directives.

Pre-processing directives are not tokens and are not part of the syntactic grammar of C#. However, pre-processing directives can be used to include or exclude sequences of tokens and can in that way affect the meaning of a C# program. For example, when compiled, the program

```
#define A
#undef B

class C
{
    #if A
        void F() {}
    #else
        void G() {}
    #endif
}
```

```

1      #if B
2          void H() {}
3      #else
4          void I() {}
5      #endif
6      }

```

7 results in the exact same sequence of tokens as the program

```

8      class C
9      {
10         void F() {}
11         void I() {}
12     }

```

13 Thus, whereas lexically, the two programs are quite different, syntactically, they are identical.

14 9.5.1 Conditional compilation symbols

15 The conditional compilation functionality provided by the `#if`, `#elif`, `#else`, and `#endif` directives is
16 controlled through pre-processing expressions (§9.5.2) and conditional compilation symbols.

17 *conditional-symbol::*
18 Any *identifier-or-keyword* except `true` or `false`

19 A conditional compilation symbol has two possible states: **defined** or **undefined**. At the beginning of the
20 lexical processing of a source file, a conditional compilation symbol is undefined unless it has been
21 explicitly defined by an external mechanism (such as a command-line compiler option). When a `#define`
22 directive is processed, the conditional compilation symbol named in that directive becomes defined in that
23 source file. The symbol remains defined until an `#undef` directive for that same symbol is processed, or
24 until the end of the source file is reached. An implication of this is that `#define` and `#undef` directives in
25 one source file have no effect on other source files in the same program.

26 The name space for conditional compilation symbols is distinct and separate from all other named entities in
27 a C# program. Conditional compilation symbols can only be referenced in `#define` and `#undef` directives
28 and in pre-processing expressions.

29 9.5.2 Pre-processing expressions

30 Pre-processing expressions can occur in `#if` and `#elif` directives. The operators `!`, `==`, `!=`, `&&` and `||` are
31 permitted in pre-processing expressions, and parentheses may be used for grouping.

32 *pp-expression::*
33 *whitespace_{opt} pp-or-expression whitespace_{opt}*

34 *pp-or-expression::*
35 *pp-and-expression*
36 *pp-or-expression whitespace_{opt} || whitespace_{opt} pp-and-expression*

37 *pp-and-expression::*
38 *pp-equality-expression*
39 *pp-and-expression whitespace_{opt} && whitespace_{opt} pp-equality-expression*

40 *pp-equality-expression::*
41 *pp-unary-expression*
42 *pp-equality-expression whitespace_{opt} == whitespace_{opt} pp-unary-expression*
43 *pp-equality-expression whitespace_{opt} != whitespace_{opt} pp-unary-expression*

44 *pp-unary-expression::*
45 *pp-primary-expression*
46 *! whitespace_{opt} pp-unary-expression*

```

1      pp-primary-expression::
2          true
3          false
4          conditional-symbol
5          ( whitespaceopt pp-expression whitespaceopt )

```

When referenced in a pre-processing expression, a defined conditional compilation symbol has the boolean value `true`, and an undefined conditional compilation symbol has the boolean value `false`.

Evaluation of a pre-processing expression always yields a boolean value. The rules of evaluation for a pre-processing expression are the same as those for a constant expression (§14.15), except that the only user-defined entities that can be referenced are conditional compilation symbols.

9.5.3 Declaration directives

The declaration directives are used to define or undefine conditional compilation symbols.

```

13      pp-declaration::
14          whitespaceopt # whitespaceopt define whitespace conditional-symbol pp-new-line
15          whitespaceopt # whitespaceopt undef whitespace conditional-symbol pp-new-line
16
17      pp-new-line::
18          whitespaceopt single-line-commentopt new-line

```

The processing of a `#define` directive causes the given conditional compilation symbol to become defined, starting with the source line that follows the directive. Likewise, the processing of an `#undef` directive causes the given conditional compilation symbol to become undefined, starting with the source line that follows the directive.

Any `#define` and `#undef` directives in a source file must occur before the first *token* (§9.4) in the source file; otherwise a compile-time error occurs. In intuitive terms, `#define` and `#undef` directives must precede any “real code” in the source file.

[*Example*: The example:

```

26      #define Enterprise
27      #if Professional || Enterprise
28          #define Advanced
29      #endif
30      namespace Megacorp.Data
31      {
32          #if Advanced
33          class PivotTable {...}
34          #endif
35      }

```

is valid because the `#define` directives precede the first token (the `namespace` keyword) in the source file.

end example]

[*Example*: The following example results in a compile-time error because a `#define` follows real code:

```

39      #define A
40      namespace N
41      {
42          #define B
43          #if B
44          class Class1 {}
45          #endif
46      }

```

end example]

A `#define` may define a conditional compilation symbol that is already defined, without there being any intervening `#undef` for that symbol. [*Example*: The example below defines a conditional compilation symbol A and then defines it again.

```

1      #define A
2      #define A

```

3 For compilers that allow conditional compilation symbols to be defined as compilation options, an
 4 alternative way for such redefinition to occur is to define the symbol as a compiler option as well as in the
 5 source. *end example*]

6 A `#undef` may “undefine” a conditional compilation symbol that is not defined. [*Example:* The example
 7 below defines a conditional compilation symbol A and then undefines it twice; although the second `#undef`
 8 has no effect, it is still valid.

```

9      #define A
10     #undef A
11     #undef A

```

12 *end example*]

13 9.5.4 Conditional compilation directives

14 The conditional compilation directives are used to conditionally include or exclude portions of a source file.

```

15     pp-conditional::
16         pp-if-section pp-elif-sectionsopt pp-else-sectionopt pp-endif
17
18     pp-if-section::
19         whitespaceopt # whitespaceopt if whitespace pp-expression pp-new-line conditional-
20         sectionopt
21
22     pp-elif-sections::
23         pp-elif-section
24         pp-elif-sections pp-elif-section
25
26     pp-elif-section::
27         whitespaceopt # whitespaceopt elif whitespace pp-expression pp-new-line conditional-
28         sectionopt
29
30     pp-else-section::
31         whitespaceopt # whitespaceopt else pp-new-line conditional-sectionopt
32
33     pp-endif::
34         whitespaceopt # whitespaceopt endif pp-new-line
35
36     conditional-section::
37         input-section
38         skipped-section
39
40     skipped-section::
41         skipped-section-part
42         skipped-section skipped-section-part
43
44     skipped-section-part::
45         skipped-charactersopt new-line
46         pp-directive
47
48     skipped-characters::
49         whitespaceopt not-number-sign input-charactersopt
50
51     not-number-sign::
52         Any input-character except #

```

53 [Note: As indicated by the syntax, conditional compilation directives must be written as sets consisting of, in
 54 order, an `#if` directive, zero or more `#elif` directives, zero or one `#else` directive, and an `#endif`
 55 directive. Between the directives are conditional sections of source code. Each section is controlled by the
 56 immediately preceding directive. A conditional section may itself contain nested conditional compilation
 57 directives provided these directives form complete sets. *end note*]

A *pp-conditional* selects at most one of the contained *conditional-sections* for normal lexical processing:

- The *pp-expressions* of the `#if` and `#elif` directives are evaluated in order until one yields `true`. If an expression yields `true`, the *conditional-section* of the corresponding directive is selected.
- If all *pp-expressions* yield `false`, and if an `#else` directive is present, the *conditional-section* of the `#else` directive is selected.
- Otherwise, no *conditional-section* is selected.

The selected *conditional-section*, if any, is processed as a normal *input-section*: the source code contained in the section must adhere to the lexical grammar; tokens are generated from the source code in the section; and pre-processing directives in the section have the prescribed effects.

The remaining *conditional-sections*, if any, are processed as *skipped-sections*: except for pre-processing directives, the source code in the section need not adhere to the lexical grammar; no tokens are generated from the source code in the section; and pre-processing directives in the section must be lexically correct but are not otherwise processed. Within a *conditional-section* that is being processed as a *skipped-section*, any nested *conditional-sections* (contained in nested `#if...#endif` and `#region...#endregion` constructs) are also processed as *skipped-sections*.

[Example: The following example illustrates how conditional compilation directives can nest:

```
#define Debug      // Debugging on
#undef Trace       // Tracing off

class PurchaseTransaction
{
    void Commit() {
        #if Debug
            CheckConsistency();
            #if Trace
                WriteToLog(this.ToString());
            #endif
        #endif
        CommitHelper();
    }
}
```

Except for pre-processing directives, skipped source code is not subject to lexical analysis. For example, the following is valid despite the unterminated comment in the `#else` section:

```
#define Debug      // Debugging on

class PurchaseTransaction
{
    void Commit() {
        #if Debug
            CheckConsistency();
        #else
            /* Do something else
        #endif
    }
}
```

Note, however, that pre-processing directives are required to be lexically correct even in skipped sections of source code.

Pre-processing directives are not processed when they appear inside multi-line input elements. For example, the program:


```

1      class Hello
2      {
3          static void Main() {
4              System.Console.WriteLine(@"hello,
5              #if Debug
6                  world
7              #else
8                  Nebraska
9              #endif
10             ");
11         }
12     }

```

13 results in the output:

```

14     hello,
15     #if Debug
16         world
17     #else
18         Nebraska
19     #endif

```

20 In peculiar cases, the set of pre-processing directives that is processed might depend on the evaluation of the *pp-expression*. The example:

```

22     #if X
23     /*
24     #else
25     /* */ class Q { }
26     #endif

```

27 always produces the same token stream (`class Q { }`), regardless of whether or not `X` is defined. If `X` is defined, the only processed directives are `#if` and `#endif`, due to the multi-line comment. If `X` is undefined, then three directives (`#if`, `#else`, `#endif`) are part of the directive set. *end example*

30 9.5.5 Diagnostic directives

31 The diagnostic directives are used to explicitly generate error and warning messages that are reported in the same way as other compile-time errors and warnings.

```

33     pp-diagnostic::
34         whitespaceopt # whitespaceopt error pp-message
35         whitespaceopt # whitespaceopt warning pp-message
36
37     pp-message::
38         new-line
39         whitespace input-charactersopt new-line

```

39 [Example: The example

```

40     #warning Code review needed before check-in
41     #if Debug && Retail
42     #error A build can't be both debug and retail
43     #endif
44     class Test {...}

```

45 always produces a warning (“Code review needed before check-in”), and produces a compile-time error if the pre-processing identifiers `Debug` and `Retail` are both defined. Note that a *pp-message* can contain arbitrary text; specifically, it need not contain well-formed tokens, as shown by the single quote in the word `can't`. *end example*

49 9.5.6 Region control

50 The region directives are used to explicitly mark regions of source code.

```

51     pp-region::
52         pp-start-region conditional-sectionopt pp-end-region

```

```

1      pp-start-region::
2          whitespaceopt # whitespaceopt region pp-message
3
4      pp-end-region::
5          whitespaceopt # whitespaceopt endregion pp-message

```

No semantic meaning is attached to a region; regions are intended for use by the programmer or by automated tools to mark a section of source code. The message specified in a *#region* or *#endregion* directive likewise has no semantic meaning; it merely serves to identify the region. Matching *#region* and *#endregion* directives may have different *pp-messages*.

The lexical processing of a region:

```

10     #region
11     ...
12     #endregion

```

corresponds exactly to the lexical processing of a conditional compilation directive of the form:

```

14     #if true
15     ...
16     #endif

```

9.5.7 Line directives

Line directives may be used to alter the line numbers and source file names that are reported by the compiler in output such as warnings and errors.

[*Note*: Line directives are most commonly used in meta-programming tools that generate C# source code from some other text input. *end note*]

```

22     pp-line::
23         whitespaceopt # whitespaceopt line whitespace line-indicator pp-new-line
24
25     line-indicator::
26         decimal-digits whitespace file-name
27         decimal-digits
28         default
29
30     file-name::
31         " file-name-characters "
32
33     file-name-characters::
34         file-name-character
35         file-name-characters file-name-character
36
37     file-name-character::
38         Any character except " (U+0022), and new-line

```

When no *#line* directives are present, the compiler reports true line numbers and source file names in its output. When processing a *#line* directive that includes a *line-indicator* that is not *default*, the compiler treats the line *after* the directive as having the given line number (and file name, if specified).

A *#line default* directive reverses the effect of all preceding *#line* directives. The compiler reports true line information for subsequent lines, precisely as if no *#line* directives had been processed.

[*Note*: Note that a *file-name* differs from a regular string literal in that escape characters are not processed; the *'\'* character simply designates an ordinary back-slash character within a *file-name*. *end note*]